CS6P05ES Project

Interim Report

Smart AAC System with Facial Expression Recognition for Autism

Name:	Yasas Pasindu Fernando
ID Number:	25026764
Date:	Wednesday, 25 March 2026
First Supervisor:	Ms. Niruni Fonseka,
Second Supervisor:	Mr. Akila Udara Akalanka

Declaration

Module: CS6P05ES **Deadline: 03/25/2026**

Module Leader: Prof. Migara Alawatta **Student ID: 25026764**

PLAGIARISM

You are reminded that there exist regulations concerning plagiarism. Extracts from these regulations are printed below. Please sign below to say that you have read and understand these extracts:

(signature:)____yasaspasindufernando@gmail.com___________ Date: 03/25/2026

This header sheet should be attached to the work you submit. No work will be accepted without it.

Extracts from University *Regulations* on Cheating, Plagiarism and Collusion

Section 2.3: "The following broad types of offence can be identified and are provided as indicative examples…

(i) Cheating: including taking unauthorised material into an examination; consulting unauthorised material outside the examination hall during the examination; obtaining an unseen examination paper in advance of the examination; copying from another examinee; using an unauthorised calculator during the examination or storing unauthorised material in the memory of a programmable calculator which is taken into the examination; copying coursework.

(ii) Falsifying data in experimental results.

(iii) Personation, where a substitute takes an examination or test on behalf of the candidate. Both candidate and substitute may be guilty of an offence under these Regulations.

(iv) Bribery or attempted bribery of a person thought to have some influence on the candidate's assessment.

(v) Collusion to present joint work as the work solely of one individual.

(vi) Plagiarism, where the work or ideas of another are presented as the candidate's own.

(vii) Other conduct calculated to secure an advantage on assessment.

(viii) Assisting in any of the above.

Some notes on what this means for students:

1. Copying another student's work is an offence, whether from a copy on paper or from a computer file, and in whatever form the intellectual property being copied takes, including text, mathematical notation and computer programs.
2. Taking extracts from published sources *without attribution* is an offence. To quote ideas, sometimes using extracts, is generally to be encouraged. Quoting ideas is achieved by stating an author's argument and attributing it, perhaps by quoting, immediately in the text, his or her name and year of publication, e.g. "$e = mc^2$ (Einstein 1905)". A *references* section at the end of your work should then list all such references in alphabetical order of authors' surnames. (There are variations on this referencing system which your tutors may prefer you to use.) If you wish to quote a paragraph or so from published work then indent the quotation on both left and right margins, using an italic font where practicable, and introduce the quotation with an attribution.

Acknowledgements

The author would like to express sincere gratitude to the project supervisor, Ms. Niruni Fonseka, at ESOFT Metro Campus for her continuous guidance, constructive feedback, and support throughout this project. Gratitude is also extended to the second supervisor, Mr. Akila Udara Akalanka, at ESOFT Metro Campus for his additional advice and support across technical and academic aspects of the work.

Special thanks are also extended to the staff at Karapitiya Teaching Hospital, Galle, for their openness to a potential collaboration for the planned pilot study and for sharing valuable clinical insights on children with autism spectrum disorder.

The author is also grateful to the parents, caregivers, and speech-language therapists who provided informal feedback on the AAC interface design. Appreciation is extended to the open-source communities behind Flutter, TensorFlow, Firebase, and MobileNetV2, as well as to the creators of the public facial expression datasets used for initial model testing.

Finally, the author would like to thank family and friends for their encouragement and support throughout this work.

Abstract

Communication impairment is one of the defining features of autism spectrum disorder (ASD), and for many children it creates barriers that extend well beyond language, affecting social participation, education, and daily quality of life. In Sri Lanka, the situation is compounded by the absence of augmentative and alternative communication (AAC) tools that support the country's two official languages, Sinhala and Tamil. The commercially available options are built for Western, English-speaking users and do not reflect the cultural or linguistic realities of Sri Lankan families.

Recent advances in deep learning have made facial expression recognition (FER) a viable addition to AAC systems, allowing emotional state to inform the communication support offered to the user. Despite this potential, the combination of FER with AAC has received limited attention in published research, particularly in low- and middle-income settings where the need is arguably greatest.

This interim report documents the design, methodology, and progress of a final year project developing an AI-powered AAC system with on-device facial expression recognition for children with autism in Sri Lanka. The system takes a dual-platform approach. Platform A offers a customisable, symbol-based AAC interface with trilingual support for children at ASD severity Level 1–2. Platform B extends this with on-device emotion detection using an EfficientNetB0 model with a CBAM attention module, deployed via TensorFlow Lite, targeting children at Level 3 and above. MobileNetV2 was initially evaluated as a baseline but was replaced after EfficientNetB0 with CBAM produced stronger validation results.

The mobile application is built in Flutter and follows an offline-first architecture, with all core data stored locally using SQLite and JSON. Firebase remains a planned option for future synchronisation, but at the interim stage backup is handled through manual export. A therapist-parent dashboard is included to support collaboration and progress monitoring. The emotion recognition component classifies six states, namely happy, sad, angry, fear, neutral, and tired, with a target accuracy of at least 80 per cent.

This report covers the background literature, system architecture, development methodology, work completed to date, and the planned remaining tasks. The objective is to produce a

working proof of concept that addresses a genuine gap in assistive technology provision for children with autism in Sri Lanka.

Keywords: augmentative and alternative communication, autism spectrum disorder, facial expression recognition, EfficientNetB0 + CBAM, TensorFlow Lite, Flutter, offline-first architecture, Sri Lanka, affective computing, assistive technology, multilingual.

Table of Contents

List of Figures

List of Tables

1. Introduction

1.1 Background and Context

Autism spectrum disorder (ASD) is a neurodevelopmental condition characterised by persistent difficulties in social communication alongside restricted, repetitive patterns of behaviour (American Psychiatric Association, 2013). Global prevalence is estimated at approximately one in 160 children (World Health Organization, 2021), although more recent studies in high-income countries report rates as high as one in 36 (Maenner et al., 2023). Communication impairment is central to the condition. The DSM-5 classifies severity across three levels, from Level 1 ("requiring support") to Level 3 ("requiring very substantial support"). Children at Level 3 often have little or no functional speech and depend heavily on augmentative and alternative communication (AAC) to express basic needs and emotions (Beukelman and Light, 2020).

In Sri Lanka, awareness of ASD has been growing. Perera et al. (2019) reported a prevalence of approximately 1.07 per cent among children aged two to nine, broadly consistent with global estimates. However, specialist services remain difficult to access, particularly outside major cities. Critically, there are no commercially available AAC tools that support both Sinhala and Tamil, the two official languages of the country (Samad et al., 2020). The shortage of trained speech-language therapists further compounds the challenge for families seeking communication support (Wickramasinghe et al., 2021).

1.2 Project Overview

The aim of this project is to design and develop a dual-platform AAC system for children with autism in Sri Lanka, with each platform mapped to a distinct clinical communication profile. The split into Platform A and Platform B was an intentional design decision informed by technical constraints and field-facing clinical feedback, including discussions with clinicians at Karapitiya Teaching Hospital. These discussions indicated that children at ASD Level 1-2 and children at Level 3 and above do not engage effectively with a single interaction model. Platform A therefore provides a simple, structured, and highly configurable AAC interface with Sinhala, Tamil, and English support for children who can engage with symbol-based communication. Platform B targets children with minimal or no functional speech by extending AAC with on-device facial expression recognition and emotion-adaptive assistance. Existing tools such as Avaz informed the baseline comparison, but clinicians and therapists highlighted limitations related to cost, flexibility, and local clinical fit, which reinforced the need for a lightweight locally aligned design.

The dual-platform design was adopted based on clinical feedback indicating that children at different ASD severity levels require distinct interface designs, particularly in terms of complexity, adaptability, and communication support mechanisms.

- **Platform A:** A customisable, symbol-based AAC interface for children at ASD severity Level 1–2, with trilingual support (Sinhala, Tamil, English), configurable vocabulary, text-to-speech output, and visual scheduling.
- **Platform B:** An AI-enhanced AAC platform for children at severity Level 3 and above, extending Platform A with on-device facial expression recognition (FER) using an EfficientNetB0-based model with a CBAM (Convolutional Block Attention Module), deployed via TensorFlow Lite, enabling emotion-adaptive communication support.

The dual-platform design was adopted based on clinical feedback indicating that children at different ASD severity levels require distinct interface designs, particularly in terms of complexity, adaptability, and communication support mechanisms.

The mobile application is developed using Flutter for cross-platform deployment, following an offline-first architecture. At the interim stage, data is stored locally on the device using SQLite and JSON, with backup handled through manual export. Firebase is planned for future optional synchronisation and account-based collaboration but is not yet implemented.

1.3 Rationale and Research Gap

A review of existing AAC systems (see Section 2.2.4) confirms that no commercially available tool combines trilingual support for Sinhala, Tamil, and English with on-device facial expression recognition. Tools such as Proloquo2Go, TouchChat, and Avaz AAC serve primarily English-speaking, Western populations. Their vocabularies, symbol sets, and cultural assumptions do not transfer well to Sri Lanka, where the food, clothing, religious practices, and family structures represented in AAC content need to be fundamentally different (Alant and Bornman, 2021). Beyond language and culture, cost is a significant barrier. Dedicated AAC devices can run to several hundred US dollars, which places them out of reach for most Sri Lankan families.

Existing AAC systems are also largely static, presenting the same vocabulary regardless of the child's emotional state. There is growing evidence that emotion-aware interfaces can improve engagement and communication outcomes (Picard, 2000), but the integration of FER into AAC tools remains largely unexplored, particularly for children with autism. Published research has examined FER and AAC as separate domains, yet no existing system brings both together within a mobile application designed for a low- and middle-income context, with multilingual support and full offline capability.

This project directly addresses that gap. The proposed system combines trilingual AAC with on-device emotion recognition, delivered through a mobile application that does not depend on internet connectivity to function.

1.4 Project Aim and Objectives

The overall aim of this project is to design and develop an AI-powered AAC system with facial expression recognition, tailored for children with autism in Sri Lanka, and to lay the groundwork for a pilot evaluation in a clinical setting.

The project objectives are as follows.

1. To design a dual-platform system architecture serving children at ASD Level 1–2 (customisable symbol-based AAC) and Level 3+ (AI-enhanced AAC with FER), with trilingual support, offline-first operation, and secure data management.

2. To implement a cross-platform mobile application using Flutter with offline-first architecture, local storage, and manual export backup, targeting Android 8.0+ and iOS 14+.

3. To train and deploy an EfficientNetB0-based FER model with a CBAM attention mechanism, targeting six emotion classes with at least 80% accuracy, deployed on-device via TensorFlow Lite with inference under 500 ms. MobileNetV2 was initially evaluated as a baseline.

4. To develop a therapist-parent dashboard for collaboration, progress monitoring, and vocabulary management.

5. To establish ethical approval protocols and a clinical partnership for a pilot study at Karapitiya Teaching Hospital, Galle.

6. To document the project in accordance with FC6P01ES module requirements using Harvard referencing.

1.5 Research Questions

The project is guided by the following research questions.

RQ1: How can a dual-platform AAC system be designed to serve children at ASD Level 1–2 and Level 3+, while maintaining trilingual support, offline-first operation, and secure data management?

RQ2: What accuracy can an on-device FER model based on EfficientNetB0 with CBAM attention achieve when trained on 2,000–5,000 images across six emotion classes, and what factors most influence performance?

RQ3: How can therapist-parent collaboration be effectively supported through a secure dashboard integrated with the mobile application, while respecting privacy and consent requirements?

RQ4: What ethical, regulatory, and practical considerations must be addressed for a pilot deployment at Karapitiya Teaching Hospital?

RQ5: To what extent does the implemented system meet its functional and non-functional requirements, and what are the key areas for improvement?

Research questions RQ1–RQ4 are addressed partially in this interim report. RQ5 will be most fully addressed in the final report following system completion and pilot evaluation.

1.6 Report Structure

The remainder of this report is organised as follows. Section 2 presents the background, including a literature review, system architecture, development methodology, AI model design, and ethical considerations. Section 3 describes the work completed to date, including design artefacts and implementation evidence. Section 4 outlines the further work planned.

Section 5 provides a progress review comparing the original plan against actual progress. Sections 6 and 7 list the references and bibliography respectively.

2. Background

2.1 Autism Spectrum Disorder

2.1.1 Definition and Diagnostic Criteria

Autism spectrum disorder (ASD) is defined in the DSM-5 as a neurodevelopmental condition with two core features, namely (1) persistent difficulties in social communication and interaction, and (2) restricted, repetitive patterns of behaviour, interests, or activities (American Psychiatric Association, 2013). The DSM-5 brought together what were previously separate diagnoses, including autistic disorder, Asperger's disorder, and PDD-NOS, into a single spectrum. This change reflected the understanding that these conditions share a common neurobiology and mainly differ in how severe the symptoms are (Lord et al., 2020).

The severity classification uses three levels based on how much support a person needs. Level 1 ("requiring support") refers to individuals with noticeable social communication difficulties. Level 2 ("requiring substantial support") applies to those with more marked deficits who have limited ability to start interactions. Level 3 ("requiring very substantial support") describes individuals with severe communication deficits and very little response to social contact (American Psychiatric Association, 2013).

Table 1: ASD Severity Levels and Communication Characteristics

Severity Level	**Social Communication**	**Typical Communication Profile**
Level 1: Requiring support	Noticeable deficits without supports, with difficulty initiating interactions	Full sentences, struggles with pragmatics (turn-taking and topic maintenance), and benefits from AAC for high-demand situations
Level 2: Requiring	Marked deficits in verbal and	Simple phrases or short

substantial support	nonverbal communication, with limited initiation	sentences, and benefits from consistent AAC support
Level 3: Requiring very substantial support	Severe deficits, very limited initiation, and minimal response to social overtures	Very limited or no functional speech, and depends heavily on AAC for basic needs, preferences, and emotions

2.1.2 Communication and Emotional Regulation in ASD

Communication difficulties in ASD can range from subtle issues, such as trouble understanding sarcasm or maintaining a conversation, to a complete absence of functional speech. Around 25 to 30 per cent of children with ASD never develop functional spoken language (Lord et al., 2020). These children are often the most underserved when it comes to communication support. Even children who do develop some speech may still struggle with expressive or receptive language and can benefit from AAC as a supplementary tool.

Emotional regulation is another significant challenge. Mazefsky et al. (2013) describe emotion dysregulation as a common feature of ASD, where children have difficulty identifying, understanding, and managing their emotions. This is directly relevant to AAC design. A system that can pick up on the user's emotional state could provide better support, for example by suggesting calming vocabulary or alerting a caregiver when the child seems distressed. This connection between emotional state and communication is one of the key reasons for integrating facial expression recognition into the proposed AAC system.

2.1.3 Epidemiology

Global prevalence estimates for ASD have risen substantially over recent decades. The World Health Organization (2021) cites a figure of roughly one in 160 children, based on a systematic review by Elsabbagh et al. (2012). More recent data from the US CDC, however, put the rate as high as 2.78 per cent (one in 36) among eight-year-old children (Maenner et al., 2023). This rise is largely attributed to broadened diagnostic criteria, greater awareness, improved screening, and changes in how services are accessed (Lord et al., 2020). In low- and middle-income countries (LMICs), prevalence data tend to be limited and probably reflect underdiagnosis rather than genuinely lower rates (Divan et al., 2021).

2.1.4 Autism in Sri Lanka

Perera et al. (2019) carried out a population-based study in Sri Lanka and found a prevalence of 1.07 per cent among children aged two to nine, which is broadly in line with global figures. Samad et al. (2020) highlighted the growing demand for services, the shortage of trained professionals, and the lack of culturally suitable tools, especially in Sinhala and Tamil. Wickramasinghe et al. (2021) suggested that scalable, technology-based interventions could help fill service gaps, but stressed the need for careful local adaptation.

In practice, the clinical pathway in Sri Lanka usually starts with a referral from primary care to a teaching hospital such as Karapitiya or Lady Ridgeway for specialist assessment. From there, children may receive speech-language therapy or applied behaviour analysis, depending on availability. However, access to these services varies a great deal across the country. The Department of Census and Statistics, Sri Lanka (2012) notes that many families live in rural areas where specialist services simply are not available, and travelling for assessments can be difficult. This is where mobile-based AAC technology could make a real difference, bringing communication support directly into homes where regular therapy is not an option (Samad et al., 2020).

2.2 Augmentative and Alternative Communication

2.2.1 Definitions and Classification

AAC refers to a range of strategies, tools, and technologies used to supplement or replace natural speech for people with complex communication needs (American Speech-Language-Hearing Association, 2022). AAC systems fall into two broad categories, namely unaided (such as gestures and sign language) and aided. Aided systems are further split into low-tech options like picture boards and communication books, and high-tech options like speech-generating devices and tablet-based apps (Beukelman and Light, 2020).

One of the most widely used low-tech approaches for children with autism is the Picture Exchange Communication System (PECS), which involves exchanging picture cards to make requests. PECS is effective for building early communication skills, but it has limitations. It relies on physical materials, the vocabulary is hard to scale, and a trained communication partner needs to be present (Ganz, 2015). High-tech alternatives such as tablet-based apps go beyond these limitations by offering dynamic displays, speech output, and vocabularies that can be easily customised.

The move from dedicated speech-generating devices, which can cost thousands of dollars, to tablet-based apps has been called a "revolution" in AAC (McNaughton and Light, 2013). Tablets are cheaper, more socially acceptable, and more widely available. That said, Dawe (2006) pointed out that the success of assistive technology depends not just on what it can do technically, but also on how simple and reliable it is, and whether families are supported in using it. These points have directly shaped the design decisions in this project.

2.2.2 Evidence Base for AAC in Autism

The evidence for AAC in autism is strong and continues to grow. Ganz et al. (2012) carried out a meta-analysis of single-case studies and found moderate to large effect sizes for AAC interventions aimed at helping children make requests, with positive results across different types of AAC and age groups. Lorah et al. (2015) reviewed studies on tablet computers as speech-generating devices and found encouraging evidence, particularly noting their portability, social acceptability, and lower cost compared to dedicated devices.

An important finding from Millar et al. (2006) is that using AAC does not hold back natural speech development. In fact, it can actually help some children develop speech by reducing frustration and giving them a way to practise communication. This has helped address a common worry among parents and clinicians that AAC might discourage children from learning to talk (Light and McNaughton, 2012; Romski and Sevcik, 2005).

Light and McNaughton (2015) identify four types of communicative competence that AAC systems should support, including linguistic (language skills), operational (ability to use the technology), social (interaction and pragmatic skills), and strategic (strategies for when communication breaks down). The proposed system addresses each of these through its structured vocabulary (linguistic), simple grid-based interface (operational), caregiver-mediated use (social), and emotion-adaptive prompting during stressful moments (strategic).

2.2.3 AAC in Multilingual and Low-Resource Contexts

Most AAC research has been carried out in high-income, English-speaking countries. This leaves a significant gap for culturally and linguistically diverse populations. Alant and

Bornman (2021) argue that implementing AAC in different cultural contexts requires adapting interaction styles, respecting local communication norms, and genuinely involving families as partners rather than just giving them technology to use.

Sri Lanka's trilingual setting, with Sinhala, Tamil, and English, creates specific challenges for AAC design. Symbols, labels, and speech output all need to work properly in each language, with text-to-speech supporting correct pronunciation and natural-sounding output (Samad et al., 2020). The cultural context matters too. Food items, clothing, religious activities, and family structures in Sri Lanka are quite different from those typically represented in Western AAC vocabularies, so local adaptation is essential.

The economic realities also shape what is practical. Internet connectivity can be unreliable in rural parts of the country (International Telecommunication Union, 2022), and many families use mid-range or budget smartphones rather than expensive devices. These constraints are why this project uses an offline-first architecture, making sure the core AAC features work without needing an internet connection.

2.2.4 Comparison of Existing AAC Systems

Table 2: Comparison of Existing AAC Systems

System	Platform	Languages	AI/Adaptive Features	Offline Support	Cost
Proloquo2Go	iOS	English, Spanish, French, others	Crescendo vocabulary with no FER	Offline core	~USD 250
TouchChat	iOS, Windows	English primarily	Word prediction with no FER	Offline core	~USD 300
LAMP Words for Life	iOS	English	Motor-planning based design with no FER	Offline core	~USD 300
LetMeTalk	Android, iOS	User-configurable	None	Partial	Free

		via TTS			
CoughDrop	Web, Android, iOS	English primarily	Basic usage logging with no FER	Limited	Subscription
Avaz AAC	iOS, Android	English, Hindi, Tamil, others	Word prediction with no FER	Offline core	~USD 100-200
Proposed System	**Android, iOS (Flutter)**	**Sinhala, Tamil, English**	**On-device FER (EfficientNetB0 + CBAM, with MobileNetV2 initially evaluated) via TFLite, with emotion-adaptive AAC**	**Full offline-first**	**TBD (pilot)**

Looking at the table above, three clear gaps stand out. First, no existing system supports both Sinhala and Tamil. Second, none of them use on-device FER for emotion-adaptive communication. Third, the proposed system's full offline-first design (with local storage and manual export backup) is particularly well suited to Sri Lanka's connectivity situation.

2.3 Facial Expression Recognition and Affective Computing

2.3.1 Theoretical Foundations

Affective computing concerns the development of systems that can recognise, interpret, and respond to human emotions (Picard, 2000). Facial expression recognition (FER) is a key subfield, focusing on automatically classifying emotions from facial images or video. The theoretical basis of FER derives largely from Ekman and Friesen (1971), who proposed a set of universal basic emotions linked to specific facial muscle movements described in the Facial Action Coding System (FACS). Although the universality of these categories has been questioned (Barrett et al., 2019), the Ekman framework remains the most widely used in computational FER due to the availability of large labelled datasets built around these categories (Li and Deng, 2020).

For the proposed system, six emotion classes have been selected, namely happy, sad, angry, fear, neutral, and tired. The "tired" class replaces "surprise" and "disgust" from the standard set, as fatigue detection is more clinically relevant when working with children who may need a simpler vocabulary or a break during a session.

2.3.2 Deep Learning and Transfer Learning for FER

Convolutional neural networks (CNNs) represent the current state of the art for FER, achieving the best results on standard benchmarks such as FER2013, AffectNet, and RAF-DB (Li and Deng, 2020; Goodfellow et al., 2015). CNNs learn hierarchical feature representations directly from pixel data, progressing from low-level features (edges, textures) to high-level abstractions that distinguish emotional expressions (LeCun et al., 2015).

Transfer learning is particularly relevant to this project, where the target dataset is relatively small (2,000–5,000 images). By using a model pre-trained on ImageNet and fine-tuning it for emotion classification, the system can leverage general visual features learned from a much larger dataset (Yosinski et al., 2014).

2.3.3 Model Architecture Selection

MobileNetV2 (Sandler et al., 2018) was initially evaluated as the base architecture due to its efficiency for mobile deployment. With 3.4 million parameters, MobileNetV2 uses depthwise separable convolutions (Howard et al., 2017) to reduce computational cost by approximately 8–9 times compared to standard convolutions, enabling real-time inference on mid-range smartphones.

However, during experimentation, EfficientNetB0 combined with a CBAM (Convolutional Block Attention Module) was found to provide better feature representation and improved validation performance. EfficientNetB0 offers a more balanced trade-off between accuracy and efficiency through compound scaling, while the CBAM attention mechanism helps the model focus on the most informative facial regions. MobileNetV2 is retained as a baseline for comparison purposes.

2.3.4 FER Challenges

Several challenges are directly relevant to the design of this system.

- **Dataset bias:** Major FER datasets are predominantly composed of adult, Western, posed faces and may not generalise well to children or non-Western populations (Barrett et al., 2019; Li and Deng, 2020). Purpose-specific data collection is planned to address this.
- **Atypical expressiveness in ASD:** Children with ASD may display reduced intensity, atypical timing, or unusual facial movements (Trevisan et al., 2018). Grossard et al. (2020) found that such children produce more ambiguous expressions, leading to higher misclassification rates.
- **Environmental variability:** Real-world conditions introduce variability in lighting, head pose, and camera quality. Data augmentation (Section 2.8.2) and robust face detection preprocessing are used to mitigate these effects.
- **Inter-individual variability:** Different children express emotions differently. Confidence thresholding and caregiver override mechanisms are incorporated into the system design to address this.
- **Privacy and ethics:** FER involving vulnerable children raises significant privacy concerns, addressed in Section 2.9.

2.4 Technology Stack

2.4.1 Flutter

Flutter is an open-source UI toolkit by Google that allows building cross-platform apps from a single Dart codebase (Flutter, 2023). For this project, its main advantages include code reuse across Android and iOS, fast development cycles through hot reload, a rich set of built-in widgets, and the ability to call native code (including TensorFlow Lite) through platform channels.

2.4.2 TensorFlow Lite

TensorFlow Lite is a lightweight framework for running machine learning models on mobile devices (TensorFlow, 2023). It supports post-training quantisation, which converts 32-bit floating-point weights to 8-bit integers. This reduces the model size by about 4 times and speeds up inference by 2 to 3 times, with only a small drop in accuracy (Jacob et al., 2018). The TFLite model is packaged with the Flutter app and called through platform channels.

2.4.3 Firebase

Firebase is considered as the future backend option for this project (Firebase, 2023). At interim stage, Firebase is **not fully implemented** for end-to-end use. The system follows an offline-first approach where core AAC data is stored locally using SQLite/JSON, and the TFLite model runs entirely on the device. Backup is currently planned as a **manual export** (for example sharing a backup file through WhatsApp or file sharing). Firebase-based authentication, Firestore, and storage are kept as planned components for later sprints when stable sync and access control are ready.

Table 3: Technology Stack Summary

Component	**Technology**	**Justification**
Mobile application	Flutter (Dart)	Single codebase for Android and iOS
AI/ML model	TensorFlow/Keras; TFLite	Transfer learning, quantisation, mobile deployment
Base architecture	EfficientNetB0 + CBAM (MobileNetV2 initially evaluated)	Lightweight, optimised for mobile
Backend database (planned)	Cloud Firestore	Planned for collaboration/sync in later stage
Authentication (planned)	Firebase Authentication	Planned for secure role-based access later
Local storage (current)	SQLite + JSON	Offline-first embedded storage and vocabulary
Text-to-speech	Platform TTS / third-party	Spoken output in Sinhala,

	API	Tamil, English
Face detection	Google ML Kit	Lightweight, on-device

2.5 System Architecture

2.5.1 Architectural Overview

The system uses a dual-platform architecture built on a shared Flutter codebase, local storage services (SQLite and JSON), and an offline-first deployment model. Platform A and Platform B share core AAC components, while Platform B introduces the FER module through TensorFlow Lite and platform-channel integration. This separation was selected because clinical feedback indicated that a single unified interface would not be sufficient across ASD severity levels. Children at Level 1-2 generally benefit from structured AAC with caregiver-driven customisation, whereas children at Level 3 and above require additional assistive intelligence to interpret emotional cues when functional speech is limited. The architecture therefore combines a common software foundation with severity-appropriate interaction design, while keeping the current implementation practical for low-connectivity settings through local processing and manual export backup. The system architecture and ER design artefacts are presented in Section 3.2.

1. Two client-facing mobile applications (Platform A and Platform B) built with Flutter.
2. An on-device FER module using TensorFlow Lite, integrated into Platform B via platform channels.
3. A local data layer (SQLite/JSON) for offline storage.
4. Manual export/import for backup and sharing between caregiver and therapist (current approach).
5. A therapist-parent web-based dashboard (prototype), with planned Firebase integration later.
6. A Firebase backend (planned) for authentication and optional synchronisation in later sprints.

The architecture follows an offline-first principle, meaning all core AAC features and emotion recognition run locally on the device. Firebase is not yet implemented; the current backup approach uses manual export/import. This design decision reflects the variable internet connectivity in many parts of Sri Lanka (International Telecommunication Union, 2022). The system architecture diagram and ER diagram, produced as design artefacts, are presented in Section 3.2.

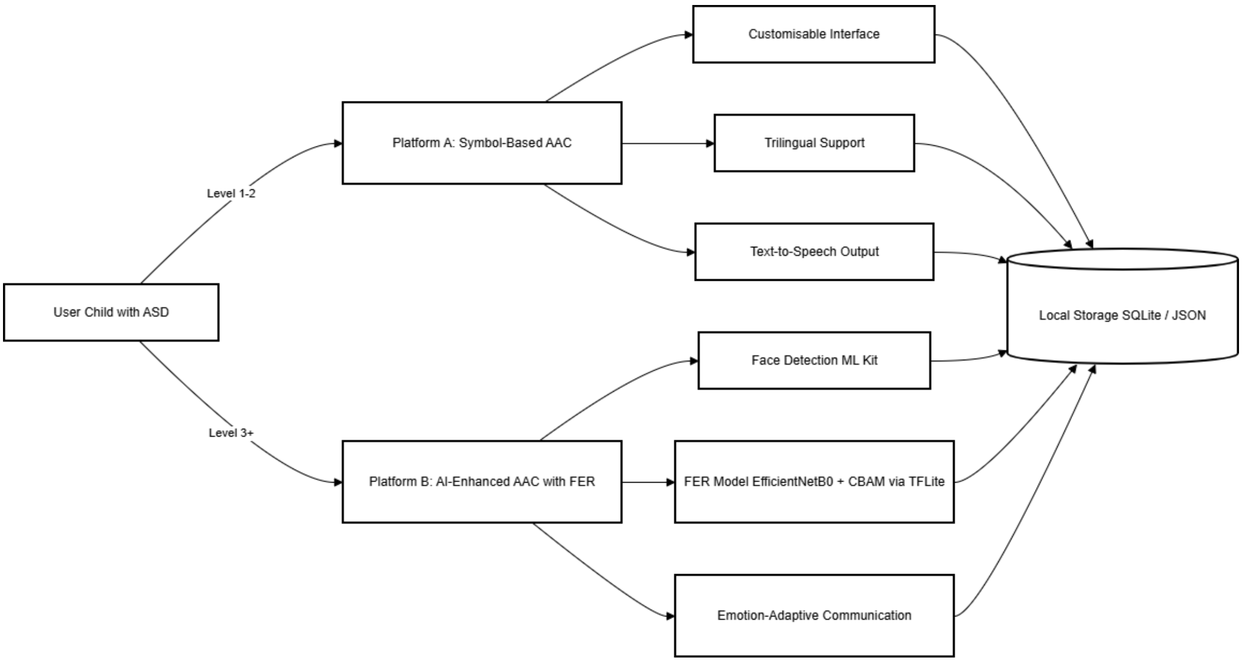


Figure 1: Dual-Platform System Architecture illustrating the separation between Platform A (Level 1-2) and Platform B (Level 3+) with local storage and core services.

2.5.2 Platform A: Customisable Symbol-Based AAC (Levels 1–2)

Platform A is designed for children at severity Level 1-2 who can interact with symbol-based communication and benefit from structured AAC support. The implementation emphasises clarity and configurability through adjustable grid sizes, trilingual language switching, text-to-speech output, caregiver and therapist configuration controls, visual scheduling, and local usage logging. This platform was included not as a secondary feature but as an essential clinical layer, because local practitioners highlighted that existing commercial tools often do not align fully with Sri Lankan language and service realities.

2.5.3 Platform B: AI-Enhanced AAC with FER (Level 3+)

Platform B is designed for children at Level 3 and above who may have minimal or no functional speech, and it represents the primary research contribution of this project. It extends Platform A with an on-device FER pipeline that analyses facial cues and supports emotion-adaptive vocabulary prompts, while preserving caregiver override at all times. This design addresses the clinical observation that highly supported children require more than a static symbol grid, and that assistive inference can improve communication guidance when direct verbal expression is limited.

2.5.4 Emotion Detection Pipeline

The emotion detection pipeline operates in five stages. The first is face detection using Google ML Kit. The second is preprocessing, which involves cropping, resizing to 224x224 pixels, and normalisation. The third is inference via the TFLite interpreter, outputting six class probabilities. The fourth is postprocessing, where argmax determines the dominant emotion, with a configurable confidence threshold below which no adaptation is triggered. The fifth is adaptation logic, which adjusts the vocabulary and notifies the caregiver, who can override the result at any time.

2.5.5 Functional Requirements

The following functional requirements have been defined for the system.

1. The system should allow caregivers and therapists to create, edit, and manage user profiles for individual children.
2. The system should display a configurable symbol grid (2x2 to 6x6) with images and text labels organised into categories such as needs, feelings, activities, and common objects.
3. The system should support switching between Sinhala, Tamil, and English for all symbol labels and interface text.
4. The system should produce spoken output of selected symbols using the device's text-to-speech engine in the active language.
5. The system should allow caregivers and therapists to add, remove, and reorder symbols, create custom categories, and adjust visual settings.

6. The system should capture facial images via the front-facing camera, process them on-device using the TFLite model, and classify the user's expression into one of six emotion classes.

7. The system should adapt the presented vocabulary based on the detected emotion, offering contextually appropriate communication options.

8. The system should allow caregivers to override, accept, or dismiss any emotion classification at any time.

9. The system should log symbol selections, session data, and aggregated emotion history locally, with the option to export data manually for backup or sharing.

10. The system should provide a therapist-parent dashboard for reviewing usage data, managing vocabulary, and monitoring progress.

2.5.6 Non-Functional Requirements

The following non-functional requirements have been defined.

1. The system should achieve emotion inference latency of less than 500 ms on a mid-range smartphone.

2. The system should start within 3 seconds of launch.

3. The system should operate fully offline for all core AAC and FER functionality.

4. The system should conform to WCAG 2.1 AA accessibility guidelines, with configurable fonts, contrast, and layout.

5. The system should not transmit raw facial images to any cloud service; only classified emotion labels and timestamps should be stored.

6. The system should encrypt any transmitted data using TLS when synchronisation is implemented.

7. The system should support Android 8.0+ and iOS 14+.

8. The system should maintain consistent and usable layouts across different smartphone screen sizes and resolutions.

Table 4: Non-Functional Requirements Summary

Requirement	Target
Emotion inference latency	Less than 500 ms on mid-range smartphone
App startup time	Less than 3 seconds
Offline operation	100% core AAC and FER available offline
Accessibility	WCAG 2.1 AA with configurable fonts, contrast, and layout
Data protection	TLS encryption with no raw facial images transmitted to cloud
Platform support	Android 8.0+ and iOS 14+
Screen-size compatibility	Responsive and readable layout across tested smartphone display sizes

2.5.7 Data Model (Interim)

At the interim stage, the main entities are stored in local storage (SQLite tables and JSON vocabulary files). The data model comprises users, categories, symbols, sessions, and logs. This structure is intended to be mapped to a cloud schema if Firebase synchronisation is implemented in a later sprint. The ER diagram is presented as a completed design artefact in Section 3.2.

2.6 Development Methodology

2.6.1 Methodology Selection

The project adopts an Agile development methodology based on an adapted Scrum framework. Several alternative methodologies were considered before arriving at this decision.

- **Waterfall model.** This was not suitable because it assumes all requirements are known upfront. That does not work well for a project involving ongoing stakeholder feedback, iterative AI model training where results are unpredictable, and external dependencies such as ethics approval whose timelines cannot be fixed in advance (Sommerville, 2016).

- **Incremental model.** The incremental model offers structured, phased delivery, but it assumes a relatively stable set of requirements from the outset. In this project, requirements have evolved continuously based on field observations at Karapitiya, feedback from caregivers, and the practical outcomes of model training experiments. The rigidity of predefined increments would not have accommodated these changes well (Sommerville, 2016).
- **Spiral model.** Boehm's (1988) spiral model focuses on risk-driven development, which is relevant given the technical and ethical risks in this project. However, its formal risk analysis cycles were considered unnecessarily complex for a single-developer final year project.
- **Rapid Application Development (RAD).** RAD focuses on speed over rigour. This conflicts with the need for proper ethical documentation, careful AI model validation, and systematic testing, all of which are essential in a healthcare-related project.

Agile Scrum was selected as the best fit for this project for several reasons.

- The nature of the project is inherently exploratory. The exact requirements for the AAC interface, the symbol vocabulary, and the FER model performance could not be fully determined at the start. Each sprint revealed new information that shaped subsequent work. For example, the field visit to Karapitiya shifted the design towards a simpler symbol grid, and early model training results led to switching from MobileNetV2 to EfficientNetB0 with CBAM.
- AI model development is iterative by nature. Training runs produce results that inform the next round of experimentation. A fixed plan cannot account for this, whereas Scrum sprints allow the backlog to be reprioritised based on the latest findings.
- External dependencies such as ethics approval and hospital coordination have uncertain timelines. Scrum accommodates this by allowing tasks to be moved between sprints without disrupting the overall framework.
- The dual-platform architecture still benefits from phased delivery, with Platform A features developed before Platform B, but within a flexible sprint structure rather than rigid predefined phases.

Since this is a single-developer project, the standard Scrum framework was adapted to suit the context. The roles of developer, product owner, and scrum master were combined and carried out by the same individual. Formal team ceremonies such as daily stand-ups were not applicable, but sprint planning was conducted at the start of each sprint and a sprint review was held with the project supervisor at the end. Retrospective notes were recorded after each sprint to identify what worked well and what needed adjustment in the following cycle. A Google Sheets spreadsheet was used to manage the product backlog, sprint backlogs, and task progress throughout the project, providing a visual record of how work was prioritised and completed across sprints. This approach was preferred over tools such as Trello or Jira for two reasons. First, the project involved a single developer, and a shared spreadsheet offered a simpler, more direct way to track items without the overhead of a full project management platform. Second, Google Sheets is accessible through its mobile application, which proved particularly useful during visits to Karapitiya Teaching Hospital and informal meetings with clinical staff. Progress updates and backlog priorities could be reviewed and confirmed on the spot without requiring access to a laptop, allowing stakeholder input to be captured immediately and reflected in the sprint plan.

2.6.2 Sprint Plan

The project is organised into five sprints aligned with the academic timeline. The first two sprints are each approximately two months in duration, while the remaining three sprints are approximately one month each, reflecting the compressed schedule required to complete all deliverables within the academic year ending in May 2026.

Sprint 1 (November to December 2025), Requirements, Architecture, and Minimal AAC. This sprint covers literature review completion, requirements analysis, system architecture design, technology stack selection, and initial Flutter project setup. The deliverable is a documented architecture and a basic application skeleton.

Sprint 2 (January to February 2026), Platform A and Firebase Backend. This sprint covers the core AAC features for Platform A (symbol grid, trilingual support, basic TTS, navigation), Firebase backend configuration, and initial dashboard implementation. The deliverable is a functional (though incomplete) AAC application and backend.

Sprint 3 (March 2026), AI Model Training and Dataset Preparation. This sprint covers dataset assembly, full model training and evaluation, TFLite export, symbol set finalisation, and ethics application submission. The deliverable is a trained and exported model ready for integration.

Sprint 4 (April 2026), Platform B Integration, Dashboard, and Pilot Preparation. This sprint covers integration of the FER pipeline into the Flutter application, emotion-adaptive vocabulary logic, dashboard completion, ethics follow-up, and pilot protocol design. The deliverable is a complete system ready for pilot deployment.

Sprint 5 (May 2026), Pilot Execution and Final Report. This sprint covers supervised pilot use, data collection and analysis, final report writing, and preparation of deliverables. The deliverable is the final report and all supporting documentation.

2.6.3 Product Backlog

The product backlog represents all required features and deliverables of the system, prioritised based on stakeholder feedback, technical dependencies, and academic milestones. This backlog was continuously refined throughout the project as new information emerged from field observations, model training results, and supervisor feedback. Items were drawn into individual sprint backlogs at the start of each sprint during sprint planning.

Priority	Backlog Item	Target Sprint
High	Literature review and requirements analysis	Sprint 1
High	System architecture and technology selection	Sprint 1
High	Flutter project setup (Android and iOS targets)	Sprint 1
High	Symbol grid, navigation, and basic AAC flow	Sprint 2
High	Trilingual UI framework (Sinhala, Tamil, English)	Sprint 2

Medium	Basic TTS integration	Sprint 2
Medium	Firebase project setup (exploratory)	Sprint 2
High	AI model pipeline setup and initial training	Sprint 2-3
High	Full model training and TFLite export	Sprint 3
High	Platform B FER integration	Sprint 3
Medium	Emotion-adaptive vocabulary logic	Sprint 3
Medium	Dashboard completion	Sprint 4
High	Ethics application and approval	Sprint 4
Medium	Pilot protocol and participant recruitment	Sprint 4
High	Pilot execution and data collection	Sprint 5
High	Final report and deliverables	Sprint 5

2.6.4 Sprint Backlogs

Each sprint backlog was derived from the product backlog during sprint planning. Tasks were selected based on priority, dependencies, and project timeline constraints. For example, Sprint 2 focused on implementing the core AAC functionality, including symbol grid navigation, multilingual support, and local data storage, while Sprint 3 prioritises AI model training and integration. The tables below summarise the backlog items for each sprint.

Sprint 1 Backlog (November to December 2025)

Task	**Status**

Complete literature review (ASD, AAC, FER, Sri Lankan context)	Done
Gather requirements from literature and informal stakeholder discussions	Done
Design system architecture (dual-platform, offline-first)	Done
Select technology stack (Flutter, TensorFlow, Firebase)	Done
Initialise Flutter project with Android and iOS build targets	Done
Produce design artefacts (architecture diagram, ER diagram, use case diagram, class diagram)	Done

Sprint 2 Backlog (January to February 2026)

Task	**Status**
Implement symbol grid interface with category navigation	Done
Implement trilingual switching framework (English complete, Sinhala/Tamil partial)	In Progress
Integrate basic text-to-speech (English functional, Sinhala/Tamil under evaluation)	In Progress
Set up local data layer (SQLite, JSON vocabulary files)	Done
Create Firebase project and explore configuration	Done
Begin AI model pipeline setup (ahead of schedule from Sprint 3)	Done

Run preliminary training experiments on public datasets	Done
Conduct initial field exposure at Karapitiya Teaching Hospital	Done
Implement basic unit and widget tests	Done
Upload APK to Google Play Console for internal testing	Done

Sprint 3 Backlog (March 2026) (Current)

Task	Status
Finalise symbol set and resolve licensing	Pending
Complete Sinhala and Tamil vocabulary	Pending
Evaluate alternative TTS engines for Sinhala/Tamil	Pending
Complete curated dataset (2,000-5,000 images)	Pending
Full model training with hyperparameter tuning	Pending
TFLite export and on-device benchmarking	Pending
Submit ethics application	In Preparation

Sprint 4 Backlog (April 2026) (Planned)

Task	Status
Integrate FER pipeline into Flutter application	Pending
Implement emotion-adaptive logic and caregiver override	Pending

Complete therapist-parent dashboard	Pending
Follow up on ethics approval	Pending
Design pilot protocol and recruit participants	Pending

Sprint 5 Backlog (May 2026) (Planned)

Task	Status
Conduct supervised pilot at Karapitiya Teaching Hospital	Pending
Collect quantitative and qualitative data	Pending
Analyse pilot findings	Pending
Write final report and compile deliverables	Pending
Prepare and deliver final presentation	Pending

Sprint management and task tracking were carried out using a Google Sheets spreadsheet, with columns for Backlog, To Do, In Progress, Review, and Done. This provided a lightweight but effective tracking mechanism suited to a single-developer workflow. The spreadsheet was also accessible via the Google Sheets mobile application, which allowed backlog items and task statuses to be reviewed or updated during hospital visits and stakeholder discussions without requiring a laptop. The sprint tracking spreadsheet is presented as Figure 20 in Section 5.4.

2.7 Research Methodology

2.7.1 Research Type

This project is classified as applied research. Unlike pure or theoretical research, which aims to generate new knowledge for its own sake, applied research focuses on solving a specific, real-world problem using existing knowledge and techniques (Kothari, 2004). In this case, the problem is the absence of a culturally and linguistically appropriate AAC system with emotion recognition for children with autism in Sri Lanka. The project applies established technologies and frameworks, including deep learning for facial expression recognition, Flutter for cross-platform mobile development, and AAC design principles from the literature, to develop a practical solution that addresses this identified gap.

Applied research was considered the most appropriate classification because the primary objective is not to advance theoretical understanding of autism, AAC, or deep learning as academic disciplines, but rather to design, build, and evaluate a working system that can be used by real children, caregivers, and therapists in a Sri Lankan clinical and home context.

2.7.2 Justification for Mixed-Methods Approach

A mixed-methods research approach was selected for this project because it involves both technical system evaluation and human-centred usability assessment, and neither a purely quantitative nor a purely qualitative strategy would be sufficient on its own. The performance of the facial expression recognition model requires objective, numerical evaluation through classification metrics, which is inherently quantitative. At the same time, the usability, acceptance, and real-world effectiveness of the AAC system can only be understood through the subjective insights and lived experiences of caregivers, therapists, and observers, which is inherently qualitative. Combining both approaches provides a more comprehensive evaluation of the system than either method could achieve alone. Creswell and Creswell (2018) argue that mixed-methods designs are particularly appropriate when a research problem requires both numerical performance data and contextual human insight, which is precisely the case here.

A convergent mixed-methods design was chosen, where quantitative and qualitative data are collected in parallel during the pilot phase and then compared to provide a more complete picture of the system's effectiveness and usability.

2.7.3 Quantitative Evaluation

The quantitative component of the evaluation serves as the primary means of objectively measuring whether the system meets its defined requirements. This component is divided into two areas.

The first is FER model performance. The trained model is evaluated using standard classification metrics, including overall accuracy, per-class precision, recall, F1-score, and a confusion matrix. These metrics are computed on a held-out test set that was not used during training. The target accuracy is at least 80 per cent across six emotion classes. If the model falls below this threshold, the results will be analysed to determine which classes are underperforming and what corrective measures (such as additional data collection or class rebalancing) are needed.

The second is system usability and user experience. During the pilot phase, structured questionnaires will be distributed to caregivers and therapists who observe or facilitate the use of the application with children. These questionnaires will use Likert scale responses (1 to 5) to measure usability, satisfaction, and system effectiveness across dimensions such as ease of use, symbol relevance, navigation clarity, response time, and overall satisfaction. The System Usability Scale (SUS) will also be applied as a standardised usability measure (Brooke, 1996). Questionnaire responses will be analysed using descriptive statistics to identify patterns in user satisfaction and areas requiring improvement.

Non-functional requirements, including emotion inference latency (target under 500 ms), application startup time (target under 3 seconds), and offline reliability, will also be measured quantitatively on test devices.

2.7.4 Qualitative Evaluation

Qualitative evaluation will be conducted through semi-structured interviews with caregivers and therapists following the pilot period. These interviews will explore user experience, perceived usefulness of the system, challenges faced during use, whether the symbol vocabulary was culturally and linguistically appropriate, how useful the emotion detection feature was perceived to be, and suggestions for improvement.

Observational notes will also be recorded during pilot sessions, documenting how children interact with the interface, which symbols are used most frequently, how caregivers intervene or support the interaction, and any behavioural responses to the emotion-adaptive features. Qualitative data will be analysed thematically to identify recurring patterns and insights that the quantitative data alone may not capture.

2.7.5 Evaluation Process and Drawing Conclusions

The overall evaluation process will follow a structured sequence. First, the FER model will be evaluated against its performance targets using the held-out test set, and the results will be documented with supporting metrics and visualisations such as the confusion matrix and training curves. Second, during the pilot study at Karapitiya Teaching Hospital, quantitative data (questionnaire responses, usage logs, SUS scores) and qualitative data (interview transcripts, observational notes) will be collected in parallel from caregivers and therapists.

Once all data has been collected, the quantitative results will be summarised using descriptive statistics and compared against the defined functional and non-functional requirements. The qualitative findings will be organised through thematic analysis, identifying common themes such as perceived ease of use, barriers to adoption, and suggestions for feature improvement.

Conclusions will be drawn by triangulating the quantitative and qualitative findings. For example, if the SUS score indicates moderate usability but interview responses reveal specific navigation difficulties, the conclusion would identify targeted improvements rather than a general assessment of success or failure. This triangulation approach is intended to produce conclusions that are grounded in measurable evidence while also reflecting the practical realities of use in the Sri Lankan context. The final report will present these conclusions alongside specific recommendations for future development and deployment.

The development follows an Agile Scrum methodology (described in Section 2.6), with the application built and tested in iterative sprints. For the AI component, training data is sourced from publicly available datasets in the first instance, with purpose-collected data planned following ethics approval.

2.8 AI Model Design and Data Collection

2.8.1 Transfer Learning Strategy

While MobileNetV2 was initially evaluated as a baseline model, the final training pipeline uses EfficientNetB0 with a CBAM attention module, as it showed better feature representation and improved validation performance.

The FER component uses EfficientNetB0 pre-trained on ImageNet as a feature extractor, and an attention module (CBAM) is added to help the model focus on useful facial regions. The transfer learning approach involves the following steps.

1. Loading EfficientNetB0 without the top classification layers, retaining the pre-trained convolutional layers.

2. Freezing the base model weights during initial training to prevent destruction of pre-learned features.

3. Adding CBAM attention and a custom classification head consisting of global average pooling, dense layers with dropout, and a six-unit output layer with softmax activation.

4. Training the classification head on the emotion dataset.

5. Optionally fine-tuning the full model with a low learning rate.

For training, mixed precision was used to reduce memory usage (float16), but float32 was kept in the final layers to avoid instability. This hybrid approach was mainly done to get better performance in low memory mode. Over time, epoch counts were also tuned (increased in experiments) to improve accuracy step by step.

2.8.2 Data Augmentation

Data augmentation increases the effective diversity of the training set by applying label-preserving transformations during training (Shorten and Khoshgoftaar, 2019).

Table 5: Data Augmentation Techniques

Technique	**Parameter Range**	**Rationale**
Horizontal flip	50% probability	Faces are approximately symmetrical
Rotation	Plus or minus 15 degrees	Simulates head tilt
Zoom	Plus or minus 10%	Simulates varying camera distances
Brightness	Plus or minus 20%	Simulates varying lighting
Contrast	Plus or minus 20%	Simulates varying image quality
Translation	Plus or minus 10% horizontal/vertical	Simulates imperfect face centering

2.8.3 Model Quantisation

Post-training quantisation converts model weights from 32-bit floating point to 8-bit integers, reducing model size by approximately 4x and improving inference speed by 2-3x with minimal accuracy loss (typically less than 1-2 percentage points) (Jacob et al., 2018). Dynamic range quantisation is applied as the default.

2.8.4 Evaluation Metrics

The model is evaluated using the following metrics.

- **Confusion matrix:** Reveals which emotion classes are most often confused.
- **Per-class precision, recall, and F1 score:** Precision measures the proportion of correct positive predictions; recall measures the proportion of actual positives correctly identified; F1 is their harmonic mean.
- **Overall accuracy:** The proportion of correct predictions to total predictions.
- **Inference time and model size:** Measured on a mid-range smartphone to verify non-functional requirements.

2.8.5 Dataset Composition

The target dataset is 2,000-5,000 labelled facial images across six emotion classes.

Table 6: Dataset Distribution by Emotion

Emotion Class	**Target Count (approx.)**	**Percentage**
Happy	350-850	~17%
Sad	350-850	~17%
Angry	300-750	~15%
Fear	250-650	~13%
Neutral	400-1000	~20%
Tired	350-900	~18%
Total	**2,000-5,000**	**100%**

Data sources include existing public datasets (FER2013, RAF-DB, AffectNet) filtered and relabelled for the target classes, and purpose-collected data (post-ethics approval) from consenting participants at Karapitiya Teaching Hospital. Data collection protocols define consent procedures, capture conditions, and labelling procedures with inter-rater reliability checks.

2.9 Ethical Considerations

2.9.1 Overview

Since this study involves vulnerable participants (children with ASD), sensitive data (facial images, usage logs), and deployment in a healthcare-related context, ethical considerations have been treated as fundamental design constraints from the start. The ethical framework draws on the Declaration of Helsinki (World Medical Association, 2013), the Belmont Report, and emerging guidelines for ethical AI deployment (Floridi et al., 2018; Jobin et al., 2019).

2.9.2 Informed Consent

Consent is obtained from the parent or legal guardian, with assent sought from the child in an accessible form where possible. The child's willingness to engage is monitored throughout; signs of distress or unwillingness are treated as withdrawal of assent. Draft consent forms and participant information sheets have been prepared in English and are included in the appendices. Sinhala and Tamil translations are planned for the next phase.

2.9.3 Privacy and Data Protection

Specific measures are outlined below.

- **On-device processing:** FER is performed entirely on the device. Raw facial images are not transmitted to any server.
- **No default image storage:** Only the classified emotion label and timestamp are logged.
- **Access control:** If Firebase is implemented later, security rules will restrict access to authorised users. For the current offline-first stage, access is mainly controlled by the device/user context.

- **Encryption:** When any data is transmitted (for example future sync), it will be protected using TLS. Firebase encryption at rest applies if Firebase storage is used later.
- **Retention policies:** Defined in the ethics protocol with secure deletion after the project analysis period.

2.9.4 Minimisation of Harm

- **Caregiver override:** Emotion recognition can be disabled or overridden at any time.
- **Confidence thresholding:** Low-confidence predictions are not acted upon.
- **Transparency:** The system presents emotion classification as a suggestion, not a certainty.
- **Voluntary participation:** Participants can withdraw at any time without penalty.

2.9.5 Institutional Approval

A pilot at Karapitiya Teaching Hospital requires approval from the hospital's institutional ethics committee and alignment with Ministry of Health research governance requirements (Ministry of Health, Sri Lanka, 2020). The ethics application includes the research protocol, consent forms, and data management plan.

Table 7: Ethical Risk Assessment

Ethical Risk	Likelihood	Impact	Mitigation
Consent not fully informed	Low	High	Mitigation measures include multilingual information sheets, child assent, and continuous monitoring during all sessions.
Privacy breach (facial images)	Low	Very High	Mitigation measures include on-device processing, no

			default image storage, and encryption for any retained data.
Misclassification of emotion	Medium	Medium	Mitigation measures include confidence thresholding, caregiver override controls, and clear communication that predictions remain suggestive.
Child distress during data collection	Low	High	Mitigation measures include trained data collectors, predefined stop criteria, and parental presence during sessions.
Delayed ethics approval	Medium	Medium	Mitigation measures include early submission and a prepared alternative evaluation plan while approvals are pending.

2.10 Risk Analysis

Table 8: Risk Assessment Matrix

Risk	**Category**	**Likelihood**	**Impact**	**Mitigation**

Emotion model accuracy below 80%	Technical	Medium	High	Mitigation measures include data augmentation, class rebalancing, and targeted hyperparameter tuning.
Poor performance on low-end devices	Technical	Medium	Medium	Mitigation measures include quantisation, lower-resolution inputs, and clear minimum device specifications.
Sync conflicts in offline-first design	Technical	Medium	Medium	Mitigation measures include a defined conflict resolution policy and comprehensive integration testing.
Delayed ethics/Ministry approval	Project	High	High	Mitigation measures include early submission and an alternative

				interim evaluation pathway.
Limited therapist/parent availability	Project	Medium	Medium	Mitigation measures include flexible scheduling and remote participation where appropriate.
Consent or data breach	Ethical	Low	Very High	Mitigation measures include strict access controls, encryption, and a documented incident response plan.
Sinhala/Tamil TTS quality insufficient	Technical	Medium	Medium	Mitigation measures include multiple TTS engines and a recorded-audio fallback approach.
Scope creep	Project	Medium	Medium	Mitigation measures include defined scope

				boundaries and regular supervisor review meetings.

The critical path runs through ethics approval, dataset collection, model training, Platform B integration, pilot execution, and the final report. The primary contingency is to proceed with model training on public data and complete Platform A independently.

2.11 Limitations and Scope

The scope of this work includes the design, development, and pilot evaluation of the dual-platform AAC system with FER in the Sri Lankan context. What falls outside the scope includes full randomised controlled trials, longitudinal studies, nationwide deployment, support for languages beyond Sinhala, Tamil, and English, and integration of modalities other than FER.

Known limitations are as follows.

1. **Dataset representativeness:** The model may not fully represent the diversity of Sri Lankan children, and performance for children with ASD may differ from neurotypical populations.
2. **Pilot constraints:** Sample size and duration will be constrained by ethics approval timelines and participant availability. Findings should be interpreted as preliminary and formative.
3. **Atypical expressiveness in ASD:** The model's ability to recognise emotions in children with ASD is an open question.
4. **TTS quality:** Text-to-speech quality for Sinhala and Tamil may vary across engines.
5. **Single-developer constraints:** As a solo project, the Scrum framework was adapted with all roles combined into one. This limits the breadth of testing, peer review, and formal usability evaluation.
6. **Device variability:** Performance may vary across devices; testing focuses on representative mid-range devices.

3. Work Completed

3.1 Summary of Progress

At the time of this interim submission, Sprint 1 has been completed in full and Sprint 2 is substantially complete. The literature review, system architecture, technology stack selection, and the initial Flutter project setup are all in place. A considerable amount of early effort went into understanding the problem domain, not only through published literature but also through informal conversations with parents and therapists. Those conversations surfaced practical realities that the literature alone did not fully convey, and they directly influenced several design decisions described in the sections that follow.

The system was implemented as a dual-platform solution consisting of Platform A and Platform B, each targeting different ASD severity levels. Platform A, designed for Level 1-2 users, was developed as a customisable symbol-based AAC interface with multilingual support and caregiver-controlled configuration. Core features such as the symbol grid, category navigation, and text-to-speech functionality were successfully implemented and tested on real devices.

Platform B, designed for Level 3 and above, extends this functionality by integrating an on-device facial expression recognition (FER) pipeline. The FER component includes face detection, image preprocessing, model inference using an EfficientNetB0-based architecture with CBAM, and emotion-adaptive response generation. Initial integration of the TensorFlow Lite model into the Flutter application was completed, and preliminary testing confirmed real-time inference capability within acceptable performance limits.

The decision to implement two separate platforms was influenced by clinical observations and initial field exposure at Karapitiya Teaching Hospital. Feedback from clinicians indicated that children at different ASD severity levels require significantly different interaction designs, particularly in terms of interface complexity and assistive support. As a result, Platform A focuses on structured and customisable communication, while Platform B introduces intelligent assistance through emotion recognition.

Some work originally planned for Sprint 3, particularly the AI model pipeline setup and preliminary training experiments, was started ahead of schedule during Sprint 2. This provides a useful head start on model development for the next phase.

The larger tasks remain ahead. Full dataset curation using purpose-collected data, complete model training and evaluation, Platform B integration with the FER pipeline, and the clinical pilot study are all planned for subsequent sprints. There have been some delays in specific areas, notably symbol set licensing, ethics coordination, and TTS quality for Sinhala and Tamil, but these are accounted for in the revised project plan (Section 5.4). Overall, the project is in a reasonable position for this stage of the academic timeline.

3.2 Requirements, Design, and Produced Artefacts

A comprehensive literature review covering ASD, AAC, facial expression recognition, and the Sri Lankan context was completed and is presented in Sections 1 and 2. Requirements for both platforms were gathered through published literature analysis, a review of existing AAC systems, and informal discussions with a speech-language therapist and two parents of children with ASD. The architecture, data flow, and technology choices have been documented, together with user roles (child, parent/caregiver, therapist) and use case mapping.

Key design decisions made during this phase include the dual-platform approach to address different ASD severity levels, the selection of Flutter for cross-platform development, the initial evaluation of MobileNetV2 for on-device FER with subsequent adoption of EfficientNetB0 with CBAM for improved feature extraction, and the offline-first architecture to accommodate areas with unreliable connectivity. Firebase remains a planned option for future synchronisation, while the current approach uses local storage with manual export backup.

To support the design phase of the project, several modelling artefacts were produced, including a system architecture diagram, an entity relationship diagram, a use case diagram, and a class diagram. These artefacts were used to clarify user interactions, define the data structure, and guide the system architecture prior to and during implementation. All figures presented in this section include appropriate captions in accordance with academic reporting standards.

Figure 2 presents the overall system architecture, showing the mobile AAC applications, the local storage layer, the on-device FER module, and the planned cloud components.

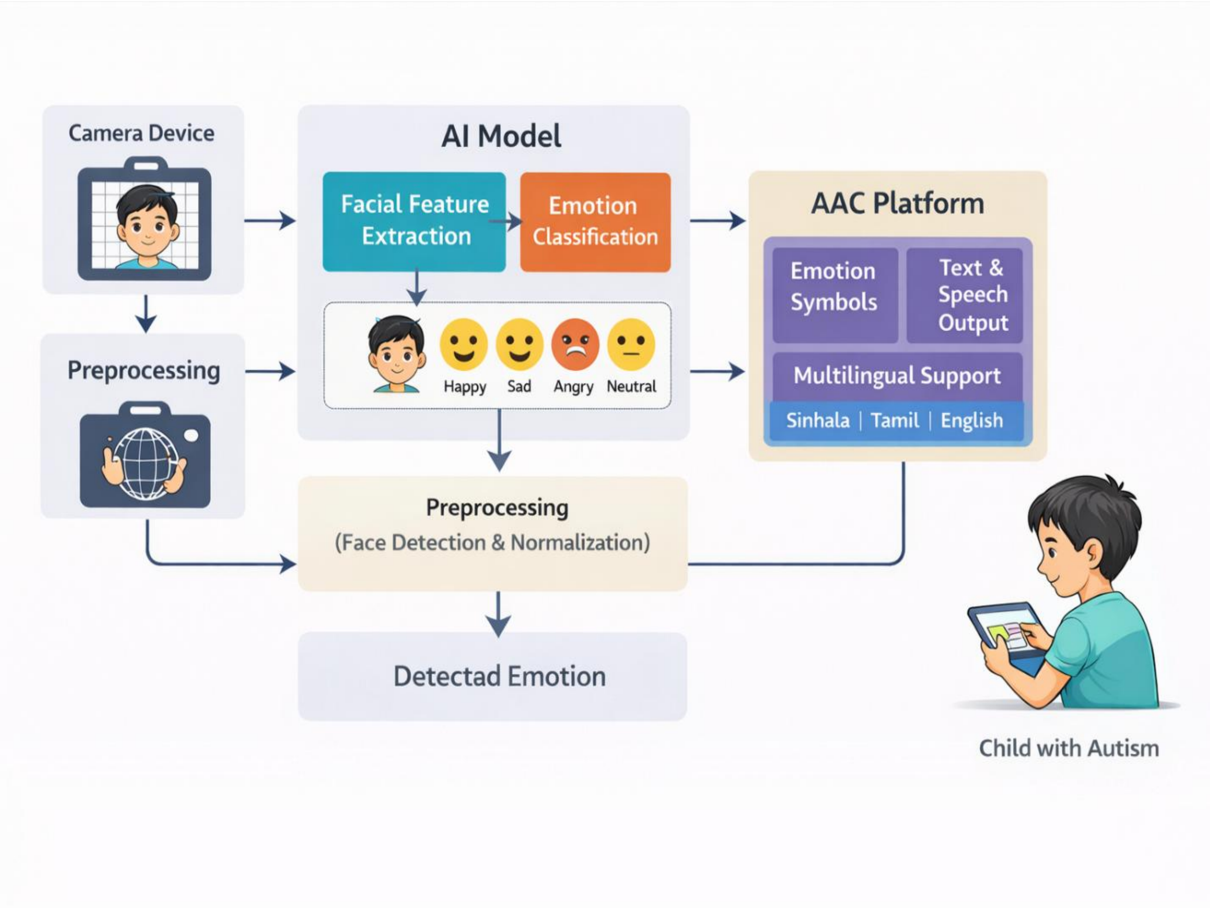


Figure 2: System Architecture Diagram

The diagram highlights the end-to-end flow from user interaction to system response. Core AAC functions are delivered locally through Platform A and Platform B, while emotion inference is processed on device through the FER pipeline in Platform B before adaptive prompts are presented. This structure supports low latency communication assistance and remains operational in environments with limited internet connectivity.

3.2.1 Entity Relationship Diagram

The entity relationship diagram represents the logical data structure of the AAC system at the interim stage. It models the key entities required to support communication, user management, and monitoring features within the application.

The system should store data related to users, child profiles, categories, symbols, sessions, usage logs, and emotion records in a structured manner. Child profiles are linked to sessions, symbol selections, and emotion records, enabling the system to maintain a consistent history of AAC interactions and detected emotional states.

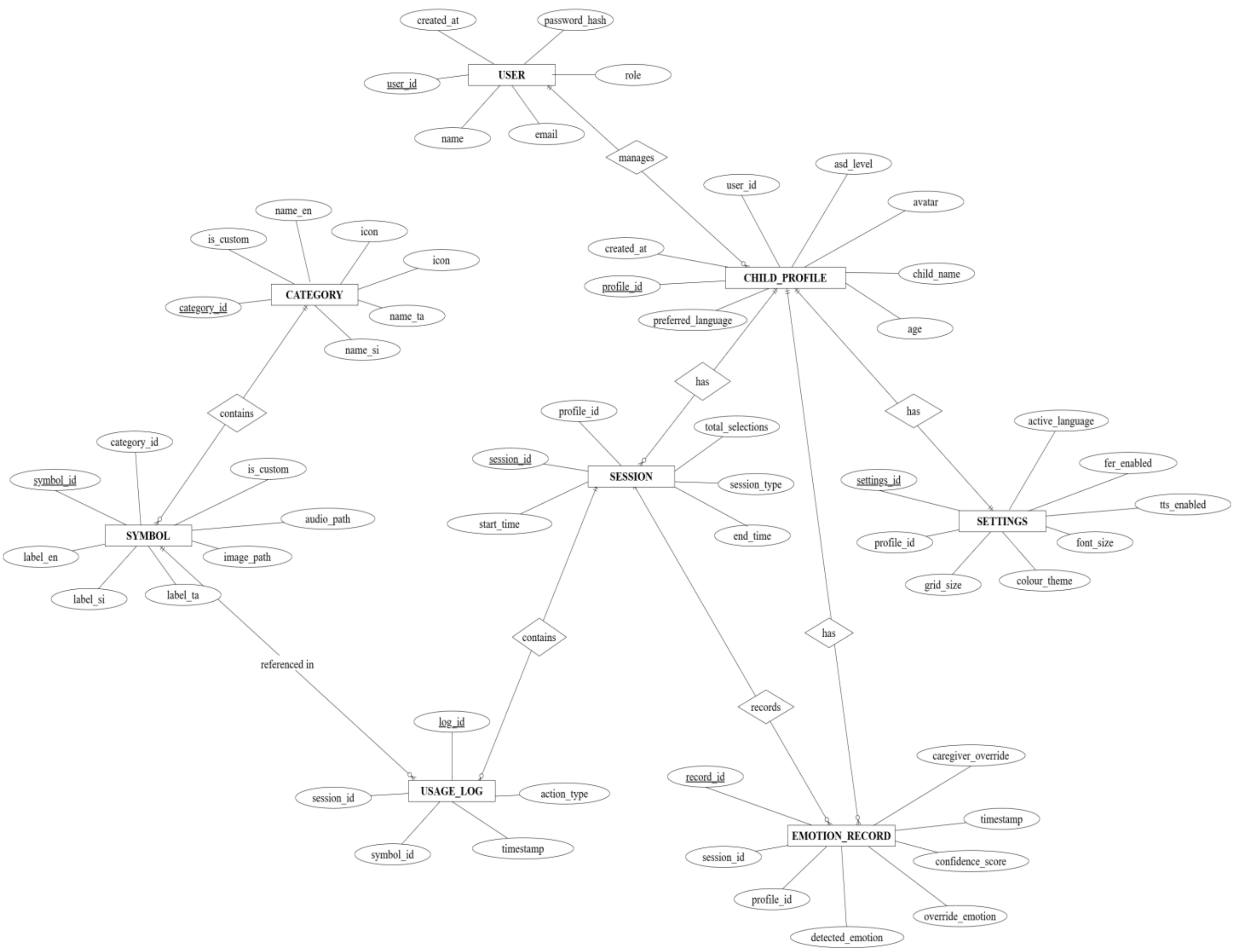
The system should support category and symbol entities to manage vocabulary in a scalable and customisable way. The system should also allow session based logging to capture user interactions over time, which can later be reviewed through the caregiver and therapist dashboard.

At this stage, the data model is implemented using local storage technologies such as SQLite and JSON files, in line with the offline first design approach. The ER diagram also provides a foundation for potential future integration with a cloud based database if synchronisation features are implemented.

Figure 3: Entity Relationship Diagram

Download Link : https://drive.google.com/file/d/1kRURwi8_up3HiGxyBgYpFTq-omLEjHHB/view?usp=sharing

ERDiagram of the AI-Enhanced AAC System



3.2.2 Use Case Diagram

The use case diagram models the primary interactions between the system and its three main user roles, namely the child, the parent or caregiver, and the therapist. It provides a high level representation of how each actor engages with the AAC system.

The child primarily interacts with communication focused features. The system should allow the child to select symbols from a grid based interface, listen to text to speech output, browse categories, view a visual schedule, and use the facial expression recognition functionality. These interactions represent the core AAC usage flow of the application.

The parent or caregiver plays both a support and management role. The system should allow caregivers to create and manage user profiles, switch between Sinhala, Tamil, and English, view detected emotions, override emotion classifications, manage symbols and categories, configure the grid size and theme, record custom audio, export and import backups, and manage child profiles.

The therapist interacts mainly with monitoring and administrative features. The system should allow therapists to access the dashboard, review usage logs and progress, view emotion history, manage vocabulary, and manage child profiles.

This diagram was used to define the functional scope of the system and to verify that all user roles and interactions had been accounted for.

Download Link :
https://drive.google.com/file/d/1iq6jvoSSK03f0fsV_psegGihgB3PPITE/view?usp=sharing

Figure 4: Use Case Diagram of the AAC System

3.2.3 Class Diagram

The class diagram describes the structural design of the AAC system at the software level by identifying the main classes and their relationships.

The system should include core classes such as User, ChildProfile, Category, Symbol, and SymbolGrid to support profile management and symbol based communication. The system should also include supporting classes such as TTSEngine, Session, and UsageLog to handle speech output and interaction tracking.

For the AI enhanced component, the system should include classes such as FERModule, FaceDetector, EmotionClassifier, AdaptationEngine, and CaregiverOverride. These classes work together to process facial input, classify emotions, and adapt the AAC interface accordingly.

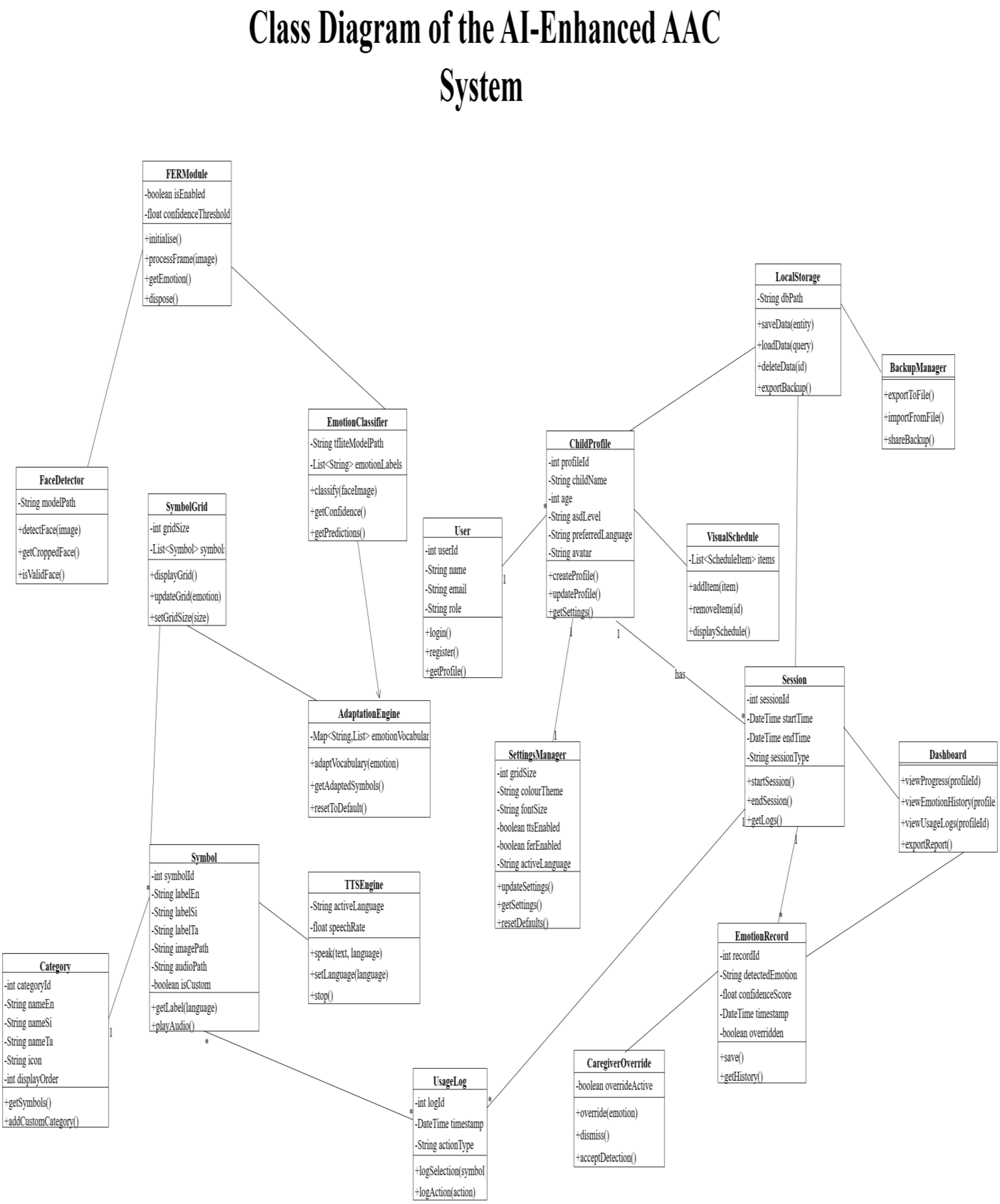
Additional classes such as LocalStorage, BackupManager, Dashboard, VisualSchedule, and SettingsManager are included to support data storage, backup handling, monitoring, scheduling, and personalisation features.

The relationships between these classes illustrate how system components interact. For example, a child profile is associated with sessions and settings, while a session contains usage logs and emotion records. The FER module depends on face detection and emotion classification, and the adaptation engine modifies the symbol grid based on the detected emotional state.

Structuring the system at this level of detail was intended to keep it modular and maintainable as development progresses.

Download Link :
https://drive.google.com/file/d/1lsY4kLsz48pxip9bFGNWLEc7vAcfNSny/view?usp=sharing

Figure 5: Class Diagram of the AAC System



3.3 Development Environment and Core AAC

The application was developed using Visual Studio Code as the primary IDE, with Flutter SDK (Dart SDK >=2.19.0) as the development framework. The project follows a standard Flutter folder structure with separate directories for models, services, screens, widgets, and utilities. It targets Android 8.0+ and iOS 14+ from a single Dart codebase.

The following table lists the key development libraries and packages used in the project, as specified in the project dependency file (`pubspec.yaml`). These packages were selected based on their relevance to the project requirements, community support, and compatibility with the offline-first architecture.

Table 9: Development Libraries and Packages

Package	**Version**	**Purpose**	**Justification for Selection**
flutter (SDK)	>=2.19.0	Cross-platform mobile framework	Single codebase for both Android and iOS, large community support, faster iteration through hot reload, and native performance through platform channels
cupertino_icons	^1.0.6	iOS-style icon set	Provides platform-consistent iconography for iOS users without requiring separate asset bundles
flutter_tts	^3.8.3	Text-to-speech output	Most widely adopted Flutter TTS package,

			supports multiple languages including Sinhala and Tamil engine access, and works offline
shared_preferences	^2.2.2	Local key-value storage	Lightweight and simple for storing user settings and preferences, has no server dependency, and supports the offline-first architecture
google_mobile_ads	^4.0.0	Advertisement integration	Planned for a future sustainability model to keep the application free for end users, and is the official Google package with strong documentation
http	^1.1.0	HTTP client	Standard Dart HTTP package for planned payment gateway and optional API calls, with a minimal footprint
connectivity_plus	^5.0.0	Network connectivity detection	Required for offline-first behaviour, and detects connection

			state changes so the application can switch between offline and online modes
camera	^0.11.0+2	Camera access	Official Flutter camera plugin that provides access to the front-facing camera required for facial expression capture in Platform B
tflite_flutter	^0.12.1	On-device ML inference	Viable option for running TensorFlow Lite models within Flutter via platform channels, and enables on-device emotion classification without cloud dependency
image	^4.1.7	Image processing	Needed for preprocessing camera frames (cropping, resizing to 224x224, normalisation) before passing them to the TFLite model
flutter_test (dev)	SDK	Unit and widget	Built-in Flutter

		testing	testing framework with no additional dependency required, and supports mock-based testing of AAC and FER logic
flutter_launcher_icons (dev)	^0.13.1	App icon generation	Automates icon generation for both Android and iOS from a single source image, and reduces manual configuration
flutter_lints (dev)	^6.0.0	Code quality	Enforces recommended Dart coding standards and best practices through static analysis

Custom assets bundled with the application include the TensorFlow Lite emotion model (`emotion_model.tflite`), emotion class labels (`labels.txt`), and the NotoColorEmoji font for consistent emoji rendering across devices.

For the AI model training component, the following Python libraries were used within Google Colab.

Table 10: AI Model Training Libraries (Google Colab)

Library	**Purpose**	**Justification for Selection**
TensorFlow / Keras	Model training and evaluation	Industry-standard deep learning framework that supports transfer learning with EfficientNetB0, and direct TFLite export for mobile deployment
NumPy	Array manipulation and computation	Required dependency for TensorFlow, and used for data preprocessing and numerical operations
Matplotlib	Training curve visualisation	Standard plotting library used to visualise loss and accuracy curves across training epochs for model evaluation
Pillow (PIL)	Image loading and preprocessing	Widely used image library that handles loading, resizing, and format conversion of facial expression images
scikit-learn	Classification metrics and confusion matrix	Provides precision, recall, F1-score, and confusion matrix functions required for model evaluation against the 80% accuracy target
tf.data API	Optimised data pipeline	Built into TensorFlow, enables efficient batching, shuffling, and augmentation during training, and reduces

		GPU idle time

Core AAC features for Platform A have been partially implemented.

- **Navigation and routing:** Screen navigation uses Flutter's Navigator 2.0 pattern, covering the main AAC grid view, category selection, settings, and profile screens.
- **Symbol grid interface:** A basic symbol grid is functional, displaying images with text labels in preliminary categories (needs, feelings, common objects). Grid size is currently fixed; configurable sizes (2x2 to 6x6) are planned for the next sprint.
- **Multilingual support:** The UI framework supports switching between Sinhala, Tamil, and English. Labels and interface text are stored in separate localisation files. The English vocabulary is mostly populated; Sinhala and Tamil vocabularies require further work.
- **Local data layer:** SQLite is used for structured data and local JSON files store vocabulary definitions, enabling full offline operation for vocabulary access and settings.
- **Text-to-speech:** Basic TTS integration using the device's built-in engine is functional for English. Preliminary testing indicated that the default platform TTS for Sinhala and Tamil produces unnatural output, and third-party TTS services will need to be evaluated.

Getting the interface right took considerably longer than the original timeline anticipated. Children with autism can disengage entirely when a screen feels cluttered or visually overwhelming (Fletcher-Watson and Happe, 2019), so every colour choice, every icon, and every tap target required deliberate consideration. Several versions were tested before the current soft palette and rounded component style was settled upon.

The symbol set proved more challenging than expected. Most established AAC symbol libraries are designed for Western contexts, and the food symbols typically show sandwiches and hamburgers, not rice, kottu, or string hoppers. A significant amount of manual sourcing and adaptation was required to build a culturally appropriate vocabulary, and licensing terms added further complexity. This work remains ongoing and has been prioritised for the next sprint.

3.4 AI Model Pipeline

Model training was conducted on Google Colab with GPU support, which proved workable but not without friction. The free tier imposes session time limits, and on several occasions training runs were interrupted before completion. In some early instances, unsaved progress was lost entirely. After those setbacks, checkpoint saving was adopted far more aggressively, which added overhead but prevented further losses.

The model architecture went through a deliberate process of iteration. MobileNetV2 was evaluated first as a baseline, being lightweight, well documented for TFLite conversion, and a reasonable starting point for mobile deployment. However, validation performance did not reach the required level. After further experimentation, EfficientNetB0 combined with a CBAM (Convolutional Block Attention Module) was selected, producing noticeably better feature representation.

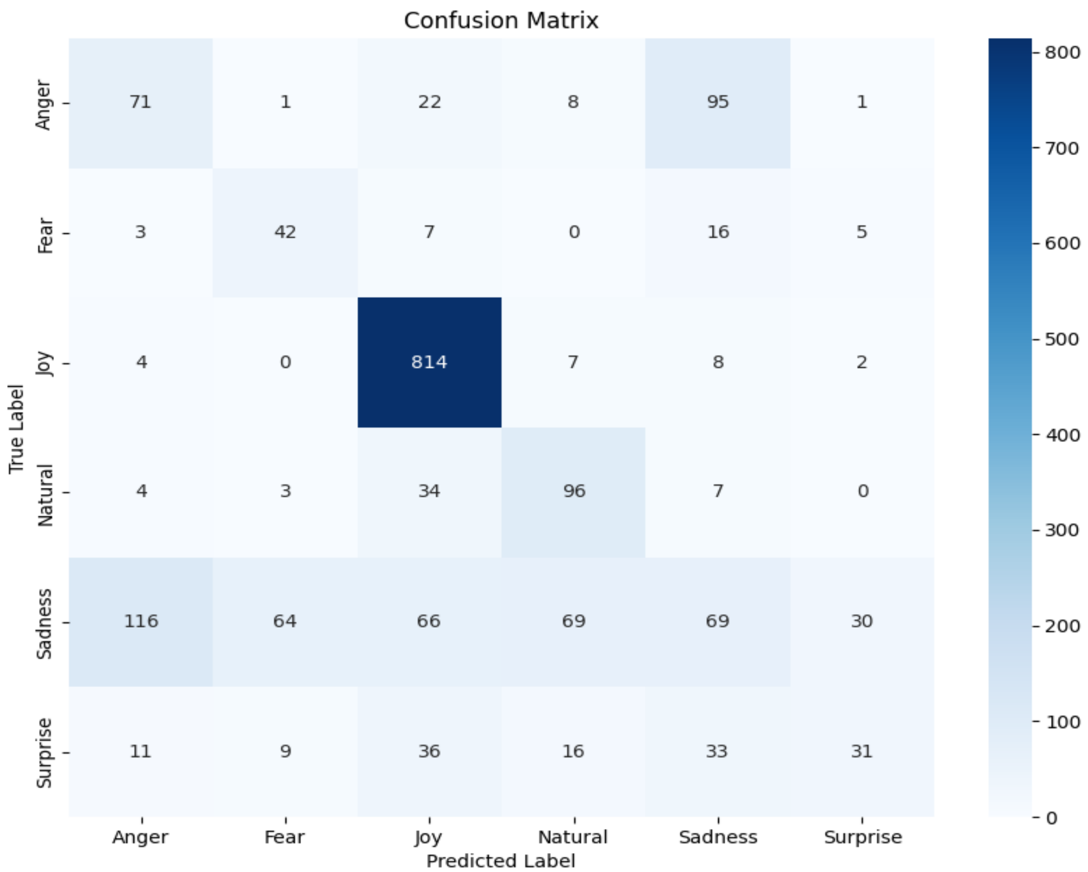
The training configuration is summarised below.

- Input size: 224x224
- Batch size: 16 (constrained by GPU memory limitations)
- Mixed precision training: float16 for speed and memory efficiency, with float32 retained in the final layers for numerical stability
- Dataset pipeline: tf.data API with optimised loading
- Class balancing: sample_from_datasets to address class imbalance
- Data augmentation: random flip, rotation, zoom, and contrast adjustments
- MixUp regularisation to improve generalisation
- Regularisation: dropout (0.4), L2 regularisation, and label smoothing
- Optimiser: AdamW
- Learning rate scheduling: ReduceLROnPlateau
- Training control: EarlyStopping

Current results show training accuracy of approximately 85–87% and validation accuracy of 82–84%, though test accuracy sits lower at around 66%. Closing that generalisation gap is the primary model objective for the next phase. The model learns the training and validation distributions reasonably well, but still struggles with unseen real world samples, which is a recognised challenge in FER when datasets are limited or imbalanced (Li and Deng, 2020). Per class analysis shows strong performance on the "happy" class, with weaker results for "fear", "sad", and "angry", most likely attributable to class imbalance and environmental variability such as lighting and head pose. Improving dataset balance and adding more diverse samples for the weaker classes will be the focus going forward.

Figure 6 presents the confusion matrix for the current FER model, illustrating which emotion classes are classified correctly and where misclassifications are concentrated.

Figure 6: Confusion Matrix

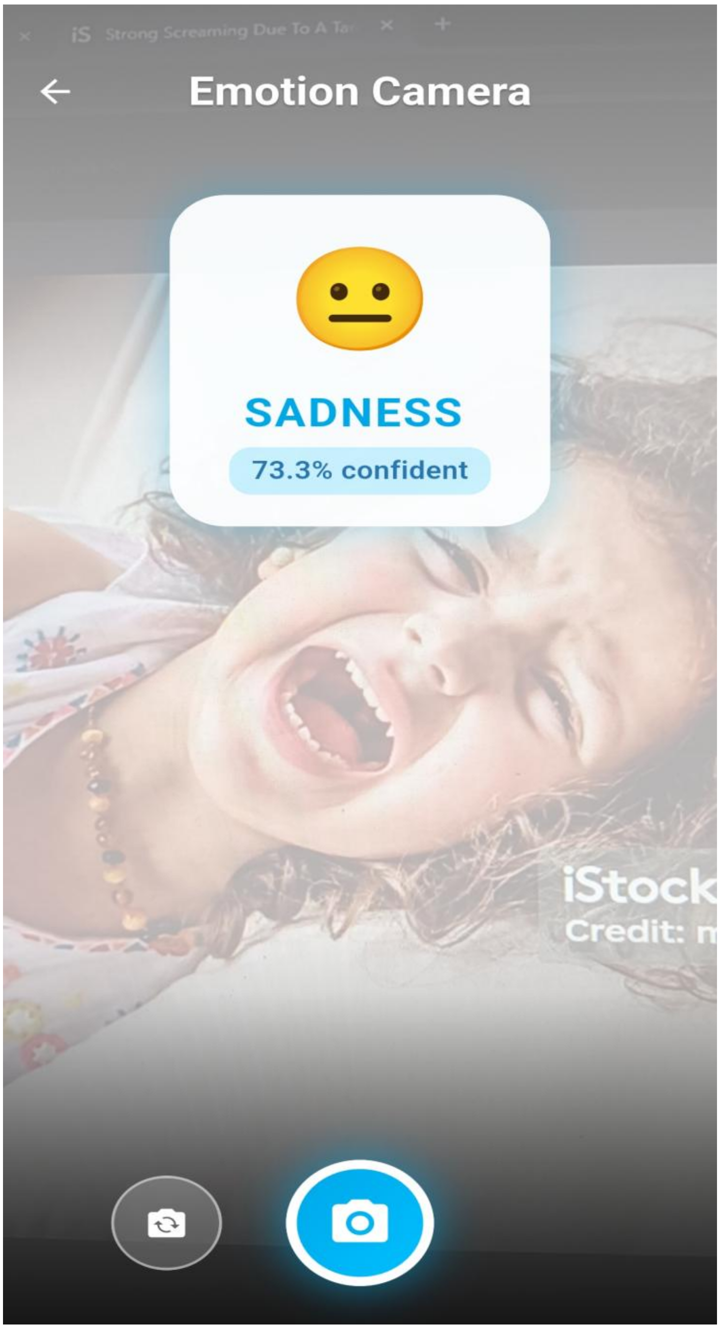


Multiple model variants were trained by adjusting epoch counts and precision settings (float32, float16, and mixed). The final model was selected based on validation performance and stability rather than training accuracy alone. Full training on the complete dataset (2,000–

5,000 images) will be conducted once the dataset is fully prepared, including purpose-collected data following ethics approval.

Figure 7 shows the facial expression recognition interface in Platform B, demonstrating on-device emotion detection using the front-facing camera and TFLite model. The detected emotion is used to adapt the AAC vocabulary accordingly, supporting contextually appropriate communication.

Figure 7: Facial Expression Recognition Screen (On-Device Detection)



3.5 Backend and Dashboard

Backend work is at an early, prototype level. A Firebase project has been created and basic configuration has been explored, but full end-to-end integration has not been completed. The current data store is local (SQLite/JSON), and backup is handled through manual export and import. A basic therapist-parent dashboard prototype has been started, including login UI and navigation screens, placeholder progress views, and an early vocabulary management interface. Full account linking and Firebase synchronisation are planned for a later stage once the offline core is stable.

3.6 Ethics and Partnership

Draft consent forms and participant information sheets have been prepared in English (included in the appendices). Sinhala and Tamil translations are planned for the next phase. Initial contact with Karapitiya Teaching Hospital has been established, and the formal ethics application is in advanced preparation. Ministry of Health approval requirements have been researched and documented.

3.7 Testing

Testing at the interim stage has focused on unit tests for the most critical parts of the codebase. The data layer functions, symbol handling logic in the AAC module, and the emotion-based decision logic in the FER integration have all been covered with automated tests. The AI model itself runs as a TensorFlow Lite component on the device, which makes direct unit testing of the full inference pipeline impractical at this stage. Instead, mock-based tests were written to validate the output handling, for example confirming that the correct dominant emotion is selected from a set of prediction probabilities.

All implemented tests were executed using Flutter's built-in testing framework and passed without failure. That said, full test coverage has not been achieved. During this phase, development priority was given to getting core features functional, with the understanding that test coverage would be expanded in subsequent sprints.

Although the application is designed for both Android and iOS platforms using Flutter, all testing conducted at the interim stage has been limited to the Android platform. This is primarily due to the unavailability of an Apple Developer account at this stage. Manual testing of the basic AAC communication flow, including symbol selection, category navigation, and text-to-speech output, has been carried out on an Android emulator and one physical Android device. Therefore, current testing results and evaluation findings are based solely on Android devices.

iOS deployment is being explored through a partnership discussion with IdeaHub (Pvt) Ltd, with the possibility of releasing the application as a charitable initiative following Ministry of Health approval. iOS-specific testing, including device compatibility, TTS behaviour, and performance validation, is planned for a later sprint once the developer account and distribution arrangements are finalised.

Formal user testing with children and caregivers will only take place after ethics approval is obtained. As described in Section 2.7, the evaluation will follow a mixed-methods approach. Quantitative evaluation will include structured questionnaires administered to caregivers and therapists, alongside model performance metrics. Qualitative evaluation will include semi-structured interviews and observational notes from pilot sessions. All evaluation evidence will be documented in the final report.

Figure 8 presents the output from the Flutter unit test suite at the interim stage.

Figure 8: Flutter Unit Test Output

```
You, 5 minutes ago |
import 'package:flutter_test/flutter_test.dart';
import 'package:aac_sinhala_tamil_english/widgets/emoji_text.dart';
import 'package:flutter/material.dart';

Run | Debug
void main() {
  Run | Debug
  testWidgets('EmojiText renders with NotoColorEmoji font',
      (WidgetTester tester) async {
    await tester.pumpWidget(
      const MaterialApp(
        home: Scaffold(
          body: EmojiText('😊', fontSize: 42),
        ), // Scaffold
      ), // MaterialApp
    );

    expect(find.text('😊'), findsOneWidget);

    final textWidget = tester.widget<Text>(find.text('😊'));
    expect(textWidget.style?.fontFamily, equals('NotoColorEmoji'));
    expect(textWidget.style?.fontSize, equals(42));
  });       YasasPasinduFernando, 3 months ago • init
}
```

test > storage_service_test.dart > ...

```dart
import 'package:flutter_test/flutter_test.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:aac_sinhala_tamil_english/services/storage_service.dart';

void main() {
  TestWidgetsFlutterBinding.ensureInitialized();

  setUp(() {
    SharedPreferences.setMockInitialValues({});
  });

  group('StorageService', () {
    test('defaults to not registered and not premium', () async {
      expect(await StorageService.isRegistered(), isFalse);
      expect(await StorageService.isPremium(), isFalse);
      expect(await StorageService.getGender(), isNull);
    });

    test('saveUserData persists registration, premium, and gender', () async {
      await StorageService.saveUserData(name: 'Test Child', gender: 'girl');

      expect(await StorageService.isRegistered(), isTrue);
      expect(await StorageService.isPremium(), isTrue);
      expect(await StorageService.getGender(), equals('girl'));
    });
  });
}
```

test > app_theme_test.dart > ...

```dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:aac_sinhala_tamil_english/screens/theme/app_theme.dart';

void main() {
  group('AppTheme', () {
    test('returns girl colors when isGirl is true', () {
      final colors = AppTheme.getThemeColors(true);
      expect(colors['primary'], equals(const Color(0xFFFF69B4)));
    });

    test('returns boy colors when isGirl is false', () {
      final colors = AppTheme.getThemeColors(false);
      expect(colors['primary'], equals(const Color(0xFF00A8E8)));
    });

    test('builds ThemeData with expected app bar color', () {
      final theme = AppTheme.getThemeData(true);
      expect(theme.appBarTheme.backgroundColor, equals(const Color(0xFFFF69B4)));
    });
  });
}
```

```
PS D:\aac_sinhala_tamil_english>
 * History restored

PS D:\aac_sinhala_tamil_english> flutter test
Resolving dependencies...
Downloading packages...
  archive 4.0.7 (4.0.9 available)
  camera 0.11.3 (0.12.0+1 available)
  camera_android_camerax 0.6.27 (0.7.1 available)
  camera_avfoundation 0.9.22+8 (0.10.1 available)
  connectivity_plus 5.0.2 (7.0.0 available)
  connectivity_plus_platform_interface 1.2.4 (2.0.1 available)
  cross_file 0.3.5+1 (0.3.5+2 available)
  dbus 0.7.11 (0.7.12 available)
  ffi 2.1.4 (2.2.0 available)
  flutter_launcher_icons 0.13.1 (0.14.4 available)
  flutter_tts 3.8.5 (4.2.5 available)
  image 4.7.1 (4.8.0 available)
  js 0.6.7 (0.7.2 available)
  json_annotation 4.9.0 (4.11.0 available)
  lints 6.0.0 (6.1.0 available)
  matcher 0.12.18 (0.12.19 available)
  meta 1.17.0 (1.18.2 available)
  petitparser 7.0.1 (7.0.2 available)
  posix 6.0.3 (6.5.0 available)
  shared_preferences_android 2.4.18 (2.4.21 available)
  source_span 1.10.1 (1.10.2 available)
  test_api 0.7.9 (0.7.11 available)
  vector_math 2.2.0 (2.3.0 available)
Got dependencies!
23 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
00:08 +6: All tests passed!
PS D:\aac_sinhala_tamil_english>
```

Given that this is an interim-stage submission and pilot activities are still pending formal completion, testing has been limited to initial automated checks. Comprehensive end-to-end, performance, and user-based testing is scheduled for the final phase.

Table 11: Work Completed Summary

Work Package	**Status**	**Notes**
Literature review	Complete	Documented in Sections 1-2
Requirements and design	Substantially complete	Documented in Section 2
Flutter project setup	Complete	Single codebase for Android/iOS
Platform A (core AAC)	Partially complete	Symbol grid, navigation, basic TTS, multilingual placeholders
Platform B (AI + FER)	In progress	Pipeline set up, with initial training on public data
Firebase backend	Early / exploratory	Project created, but not fully implemented or synced
Therapist-parent dashboard	Early / prototype	Basic UI and placeholders available, with integration pending
Ethics and partnership	In progress	Draft consent forms completed, and initial hospital contact established
AI model training	In progress	Initial experiments completed, with full training still pending
TFLite export and benchmarking	Complete (preliminary)	Inference time acceptable
Testing	In progress	Unit tests and manual testing completed, with no formal user testing yet

3.8 Current UI Screens (Interim)

The following figures present the current state of the mobile application's user interface. The interface follows a child-friendly design approach using soft colours, rounded components, and clear icon-based navigation, intended to minimise cognitive load while supporting intuitive symbol selection.

Figure 9: Register Screen (UI)

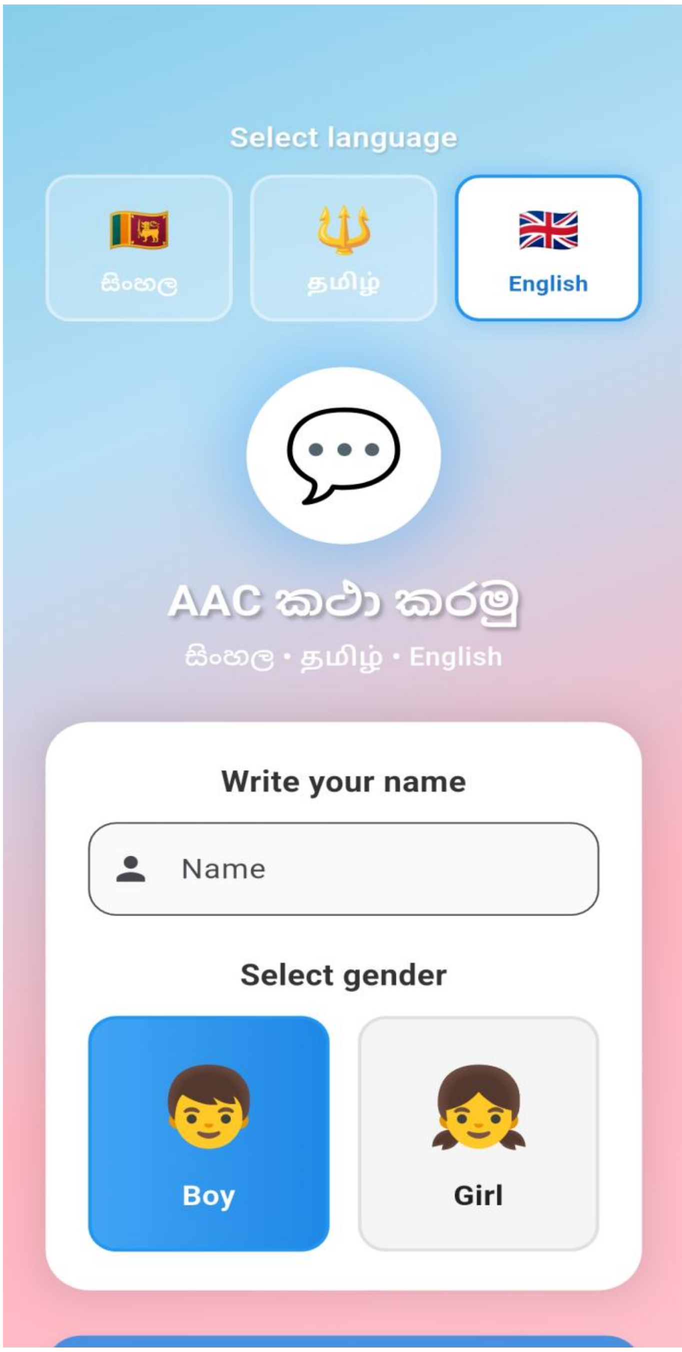
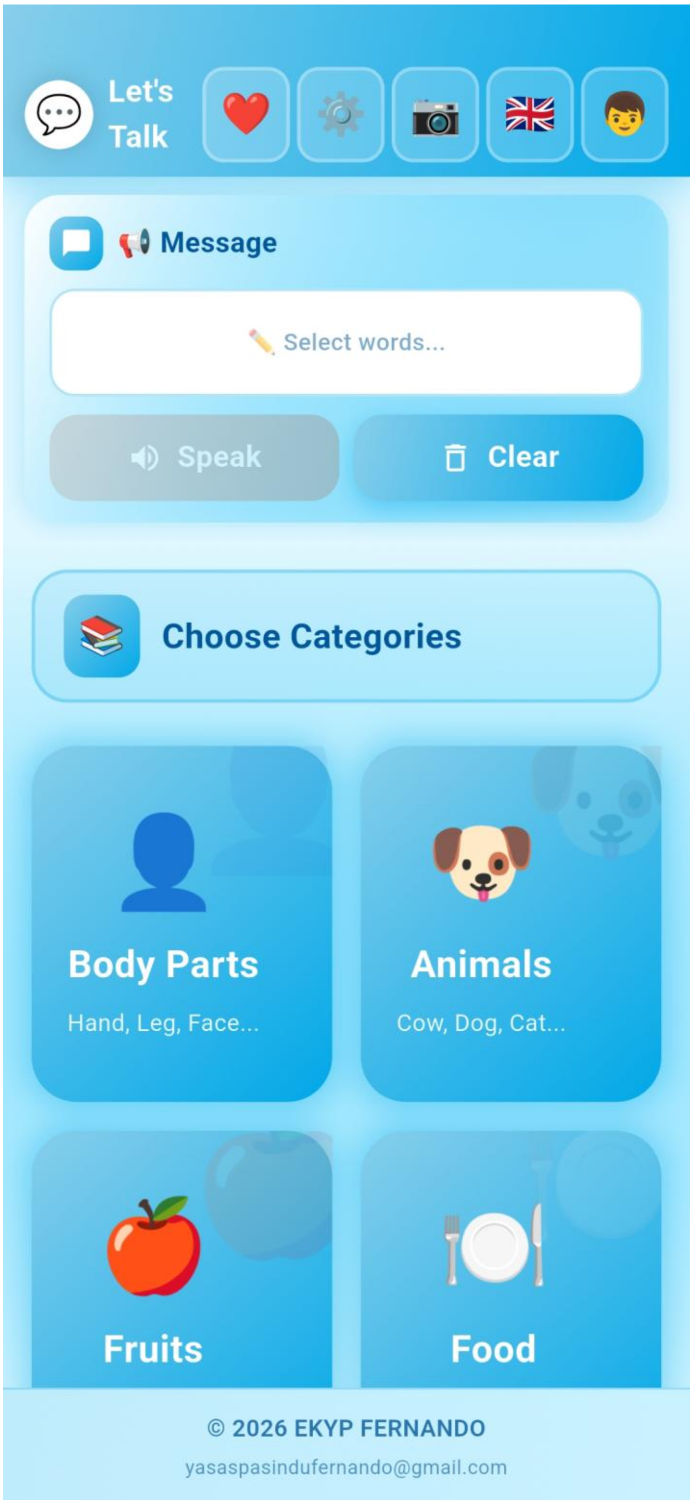


Figure 10: Home Screen (UI)

Platform B: -



Platform A: -

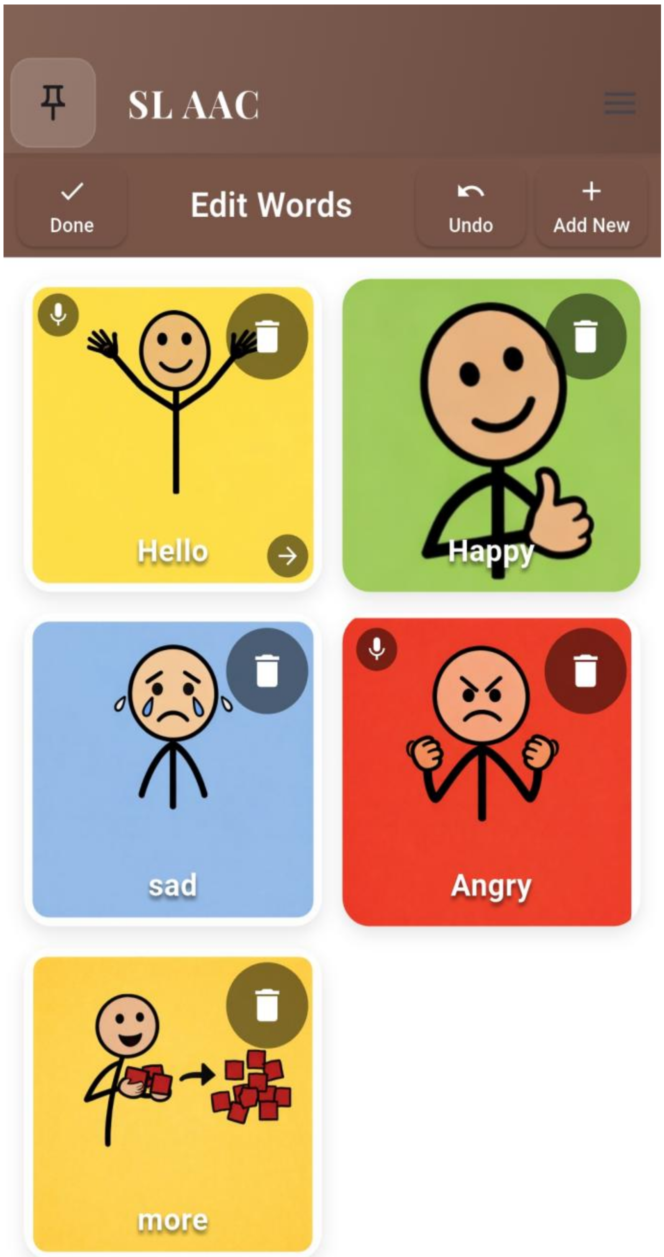
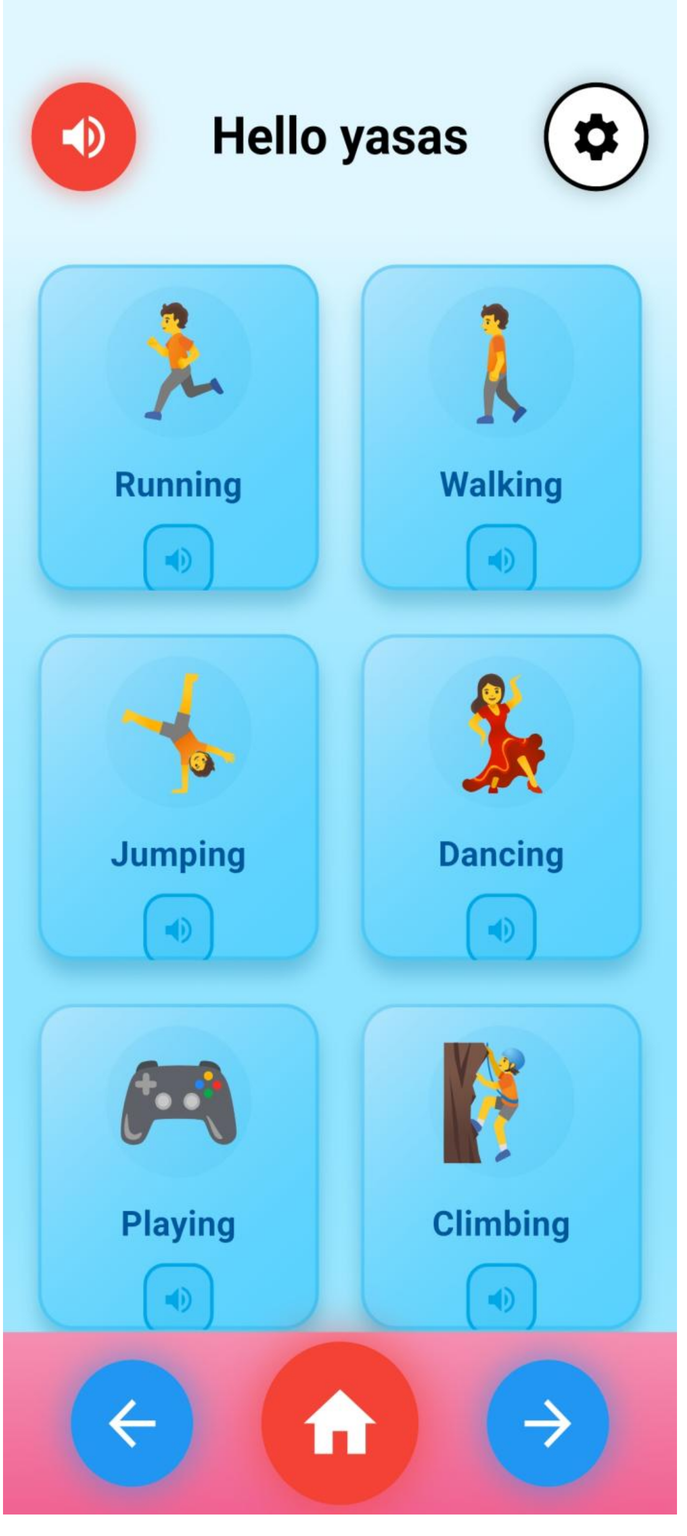


Figure 11: Categories Screen (UI)



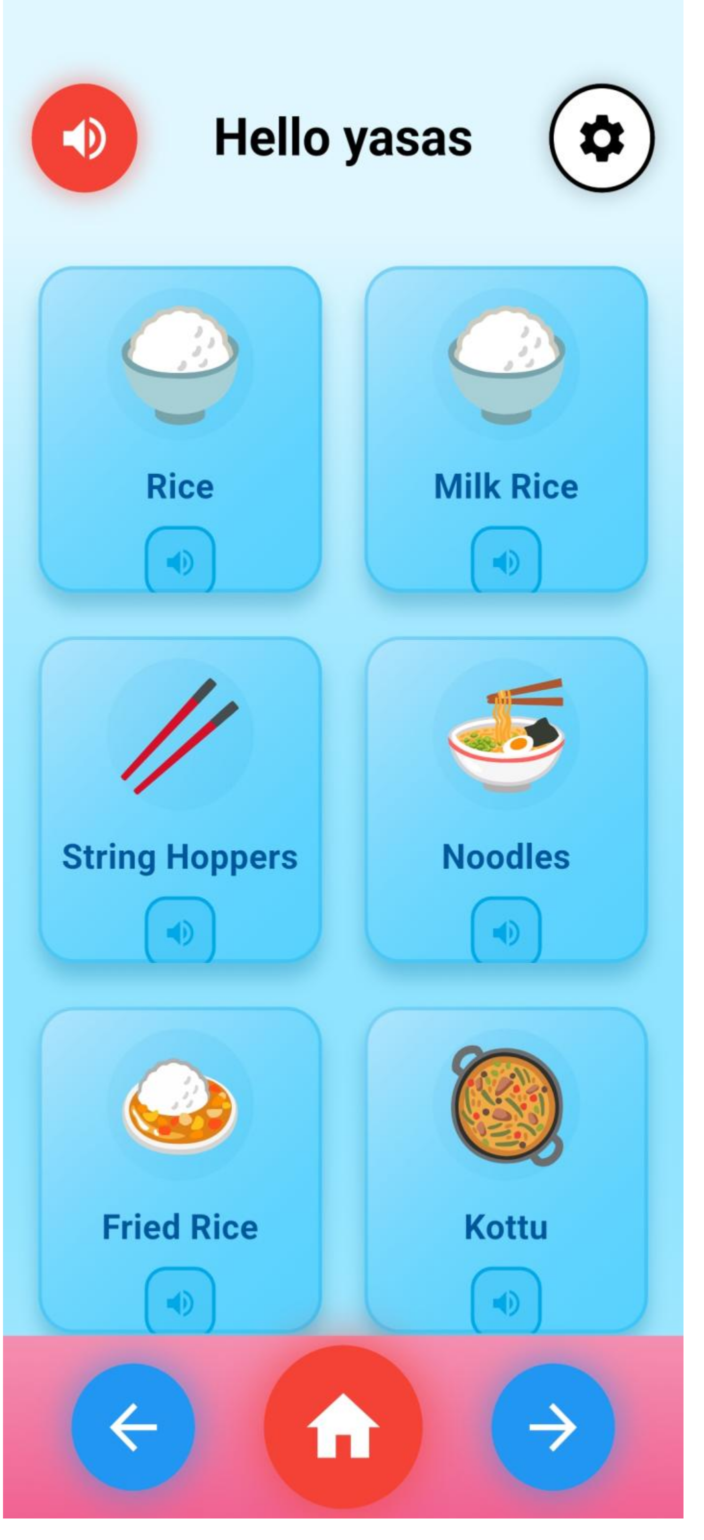


Figure 12: Settings Screen (UI)

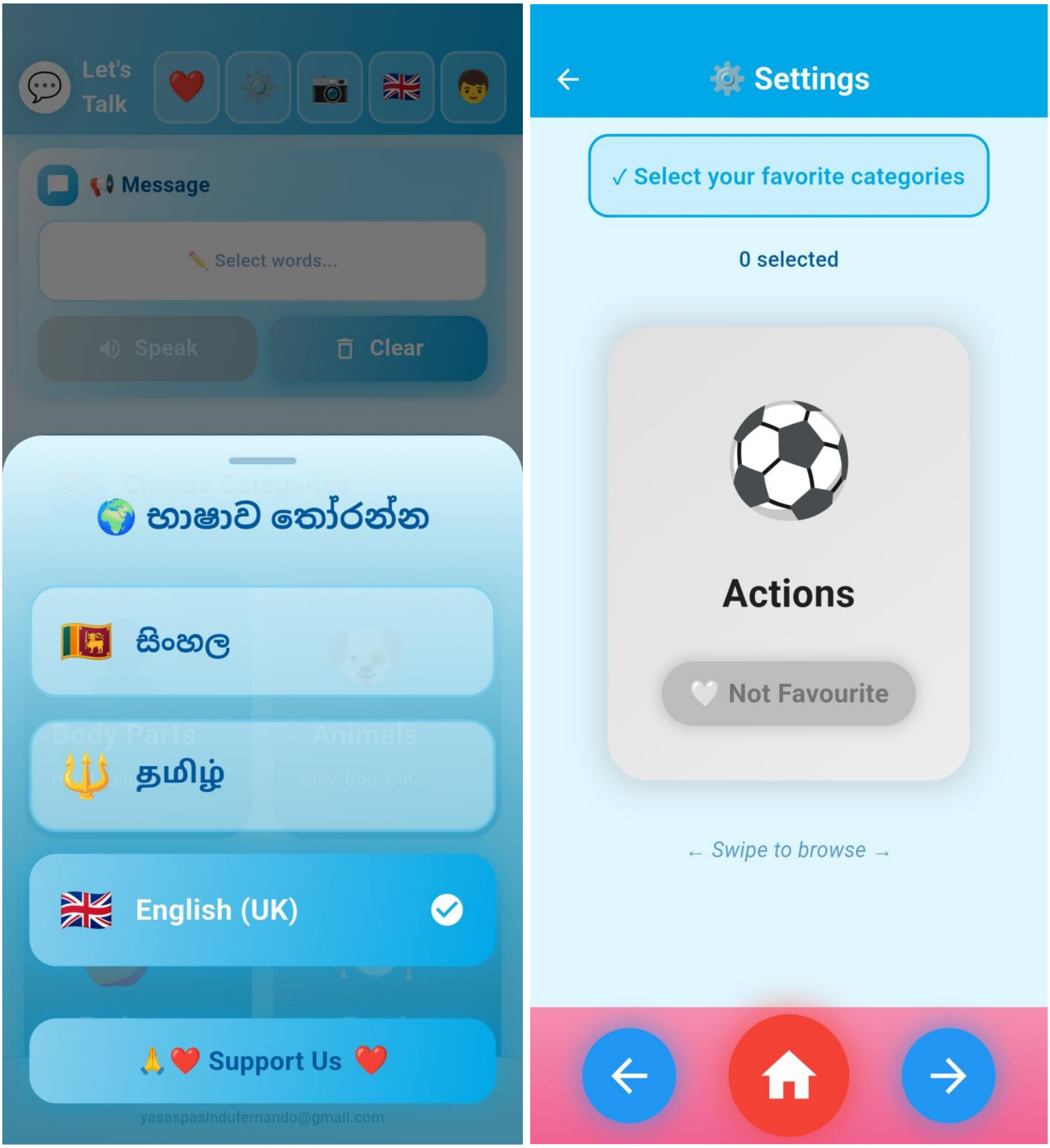
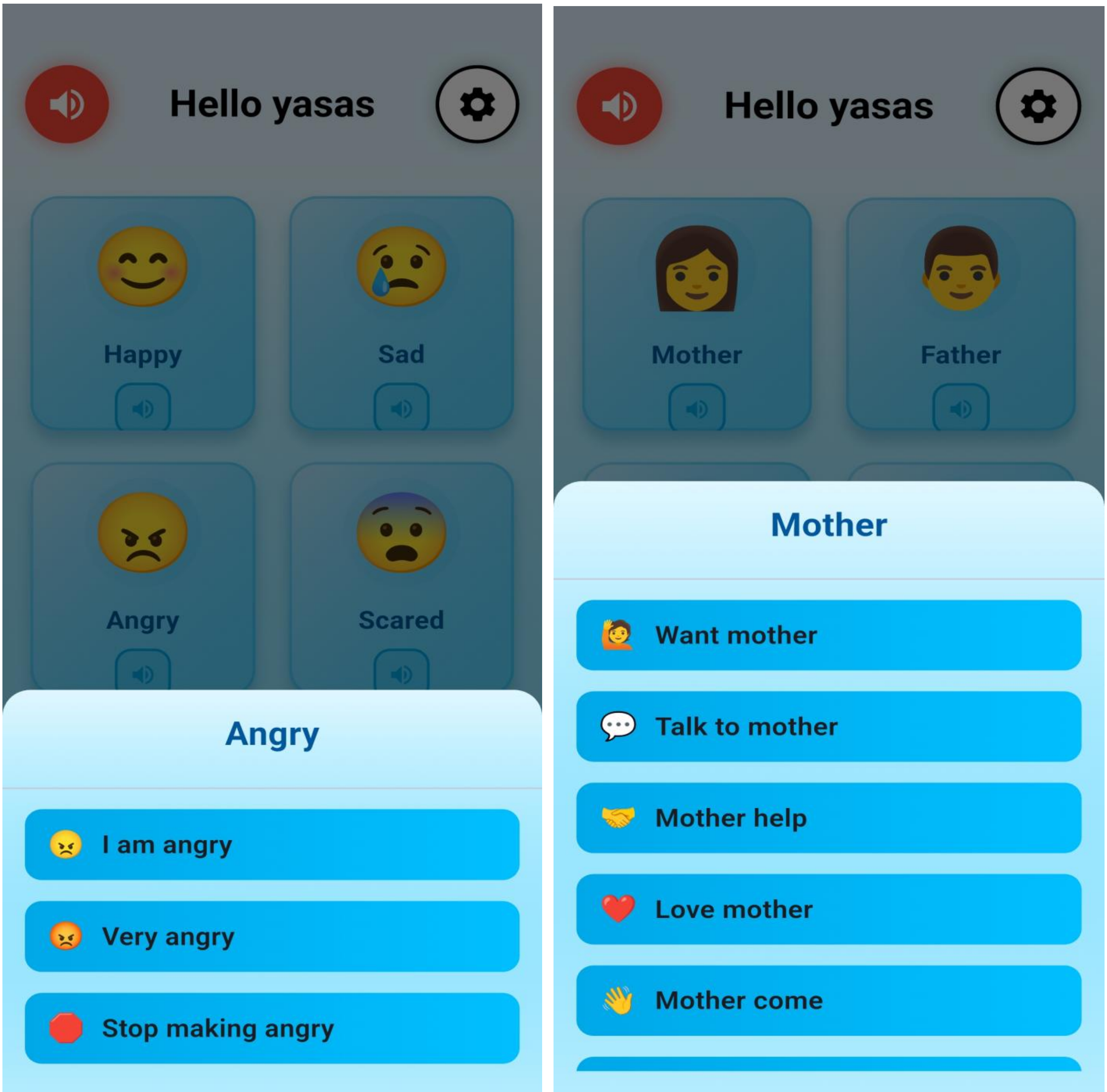


Figure 13 demonstrates a typical AAC interaction, showing symbol selection and the resulting text-to-speech output.

Figure 13: AAC Interaction Example (Symbol Selection and Output)



This figure demonstrates how a child interacts with the AAC grid, selects a symbol, and receives immediate audio feedback. This flow is designed to be simple and consistent, supporting faster learning and communication.

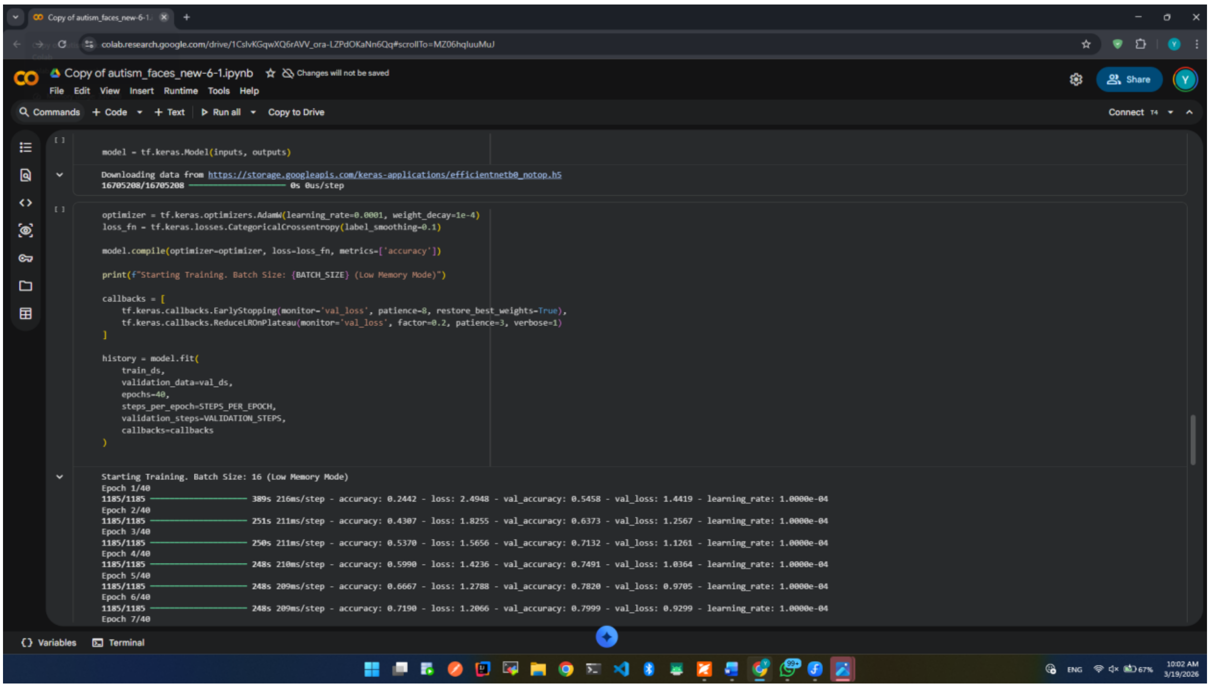
Additional user interface screens related to Platform A (Level 1-2) are provided in Appendix A. The main report emphasises Platform B due to its central role in the research, particularly the integration and evaluation of facial expression recognition.

3.9 Model Training Evidence

Model training for the FER component was conducted on Google Colab with GPU support. Multiple training runs were carried out with different combinations of epochs, learning rates, and regularisation settings. Each run was logged, and training/validation loss and accuracy curves were monitored to identify overfitting and guide parameter adjustments.

Figure 14 presents a representative Colab training session showing the loss and accuracy logs across epochs.

Figure 14: Model Training in Google Colab



Link :- https://colab.research.google.com/drive/1CslvKGqwXQ6rAVV_ora-LZPdOKaNn6Qq#scrollTo=MZ06hqIuuMuJ

The Colab notebook shows the training and validation loss/accuracy curves for each run. Mixed precision training was enabled to speed up training and reduce memory usage, while keeping key parts in float32 for stability. Across experiments, accuracy improved gradually as the number of epochs and other parameters were tuned.

During testing, monitoring these logs during runs was helpful for spotting when the model started to overfit and when to stop or adjust the epochs.

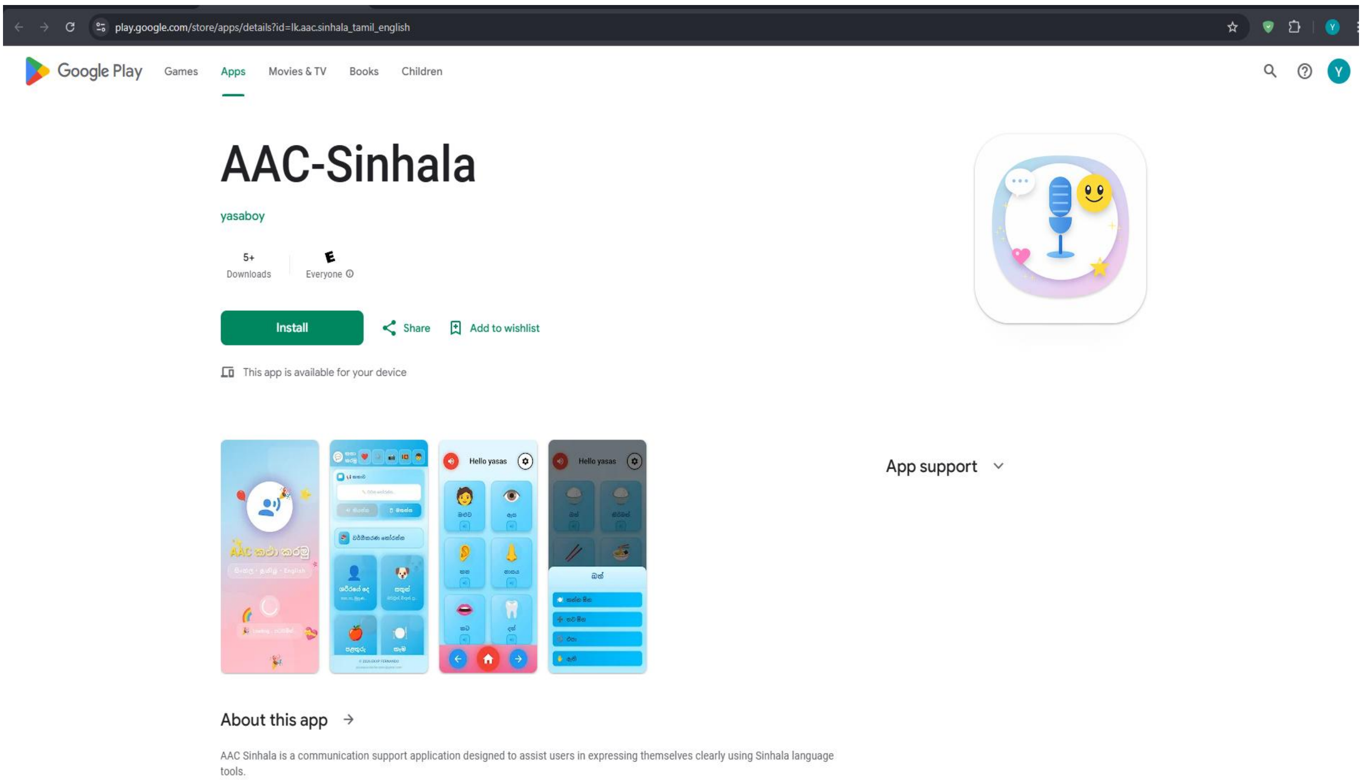
Mixed precision training (float16 for computational layers, float32 for final layers) reduced memory usage and enabled training with reasonable batch sizes on the free-tier GPU. Accuracy improved gradually across experiments as hyperparameters were refined.

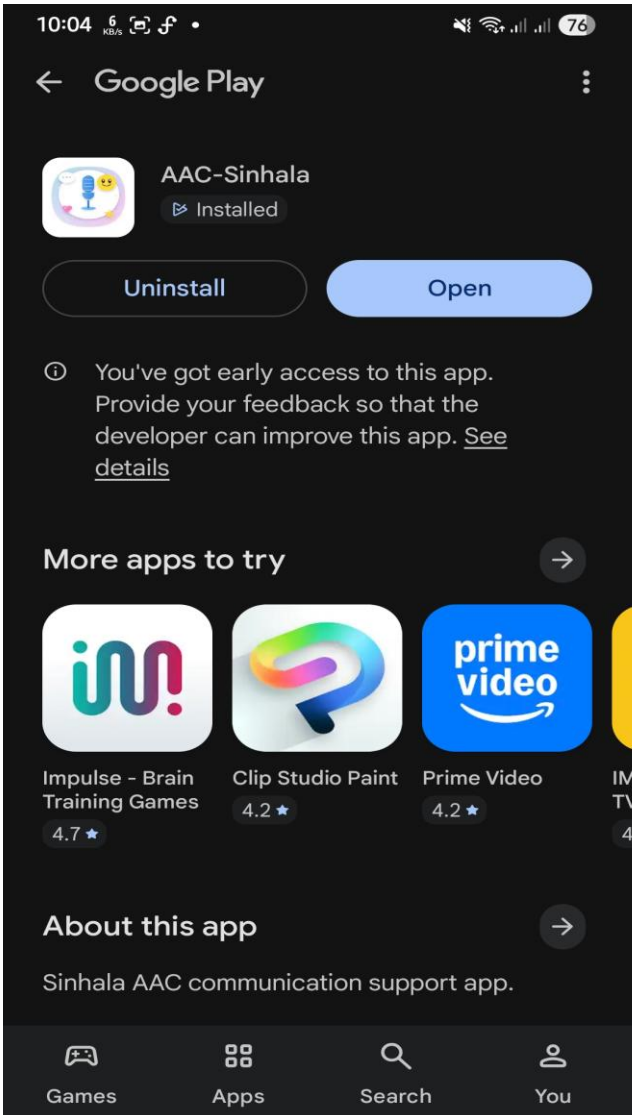
3.10 Deployment and Play Store Testing

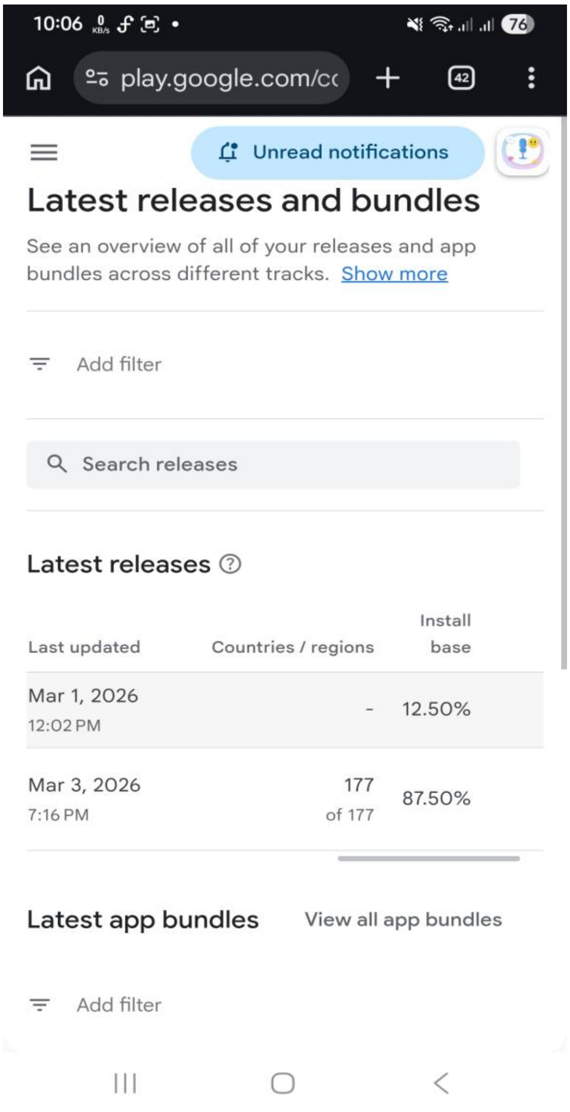
The application was packaged and uploaded to the Google Play Console under the Internal Testing track. An Android App Bundle was generated from the Flutter project and distributed to internal testers. Internal testers installed the application on their own Android devices and verified the main flows, including the AAC grid, navigation, and basic settings. This confirmed that deployment through the Play Store is technically feasible at this stage.

Figure 15 presents the Google Play Console internal testing dashboard.

Figure 15: Google Play Console Internal Testing







Play Store Link :- https://play.google.com/store/apps/details?id=lk.aac.sinhala_tamil_english

Internal testers installed the app on their own Android devices and checked the main flows, especially the AAC grid, navigation, and basic settings. This helped to confirm that the app can be installed and launched through a normal Play Store flow. A proper Play Store link will be added in the final version after further testing and polishing.

These internal tests mainly showed that, at this stage, deployment through the Play Store route is technically feasible even though the app is still in a pilot state.

iOS deployment has not yet been undertaken, as an Apple Developer account is not currently available. Discussions are ongoing with IdeaHub (Pvt) Ltd regarding the possibility of providing the developer account and distribution infrastructure for iOS, potentially as part of a charitable initiative following Ministry of Health approval.

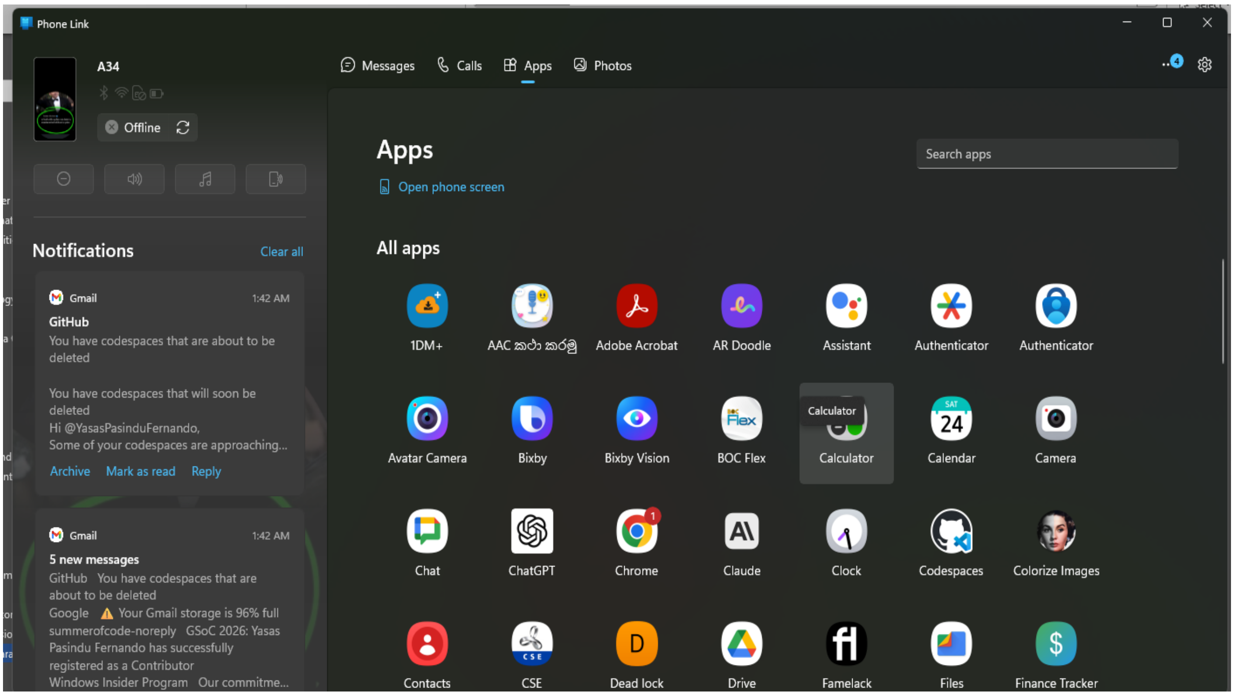
3.11 Real Device Testing Sessions

In addition to emulator testing, the application was tested on two physical Android devices under everyday conditions. These sessions focused on basic AAC communication and general stability rather than formal user studies. Symbol selection, text-to-speech output, and screen navigation were tested repeatedly to check for crashes or performance issues. Initial checks of the FER pipeline confirmed that the camera feed and on-device inference could run without freezing. Compatibility checks across different display sizes were also carried out to confirm that key interface elements remained readable and usable without layout breakage.

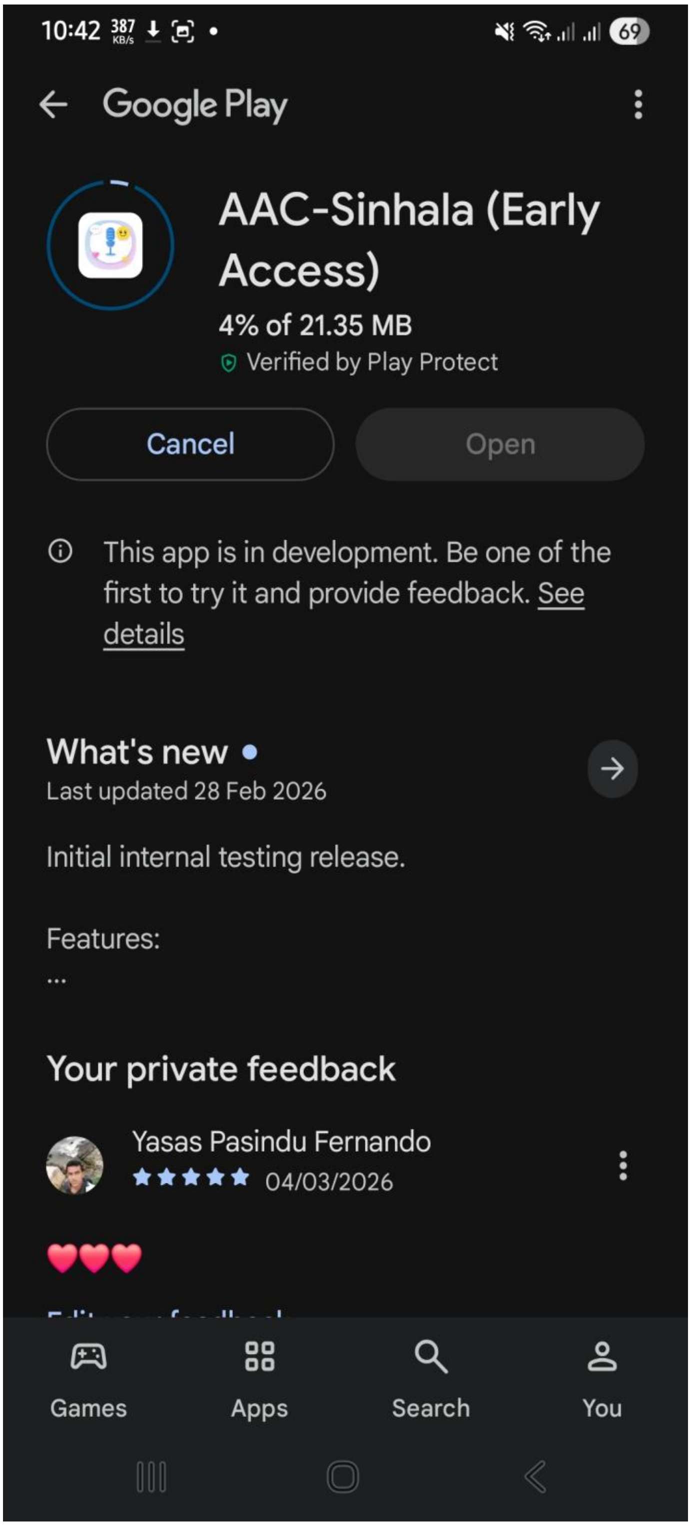
Figure 16 shows the application running on a real Android device.

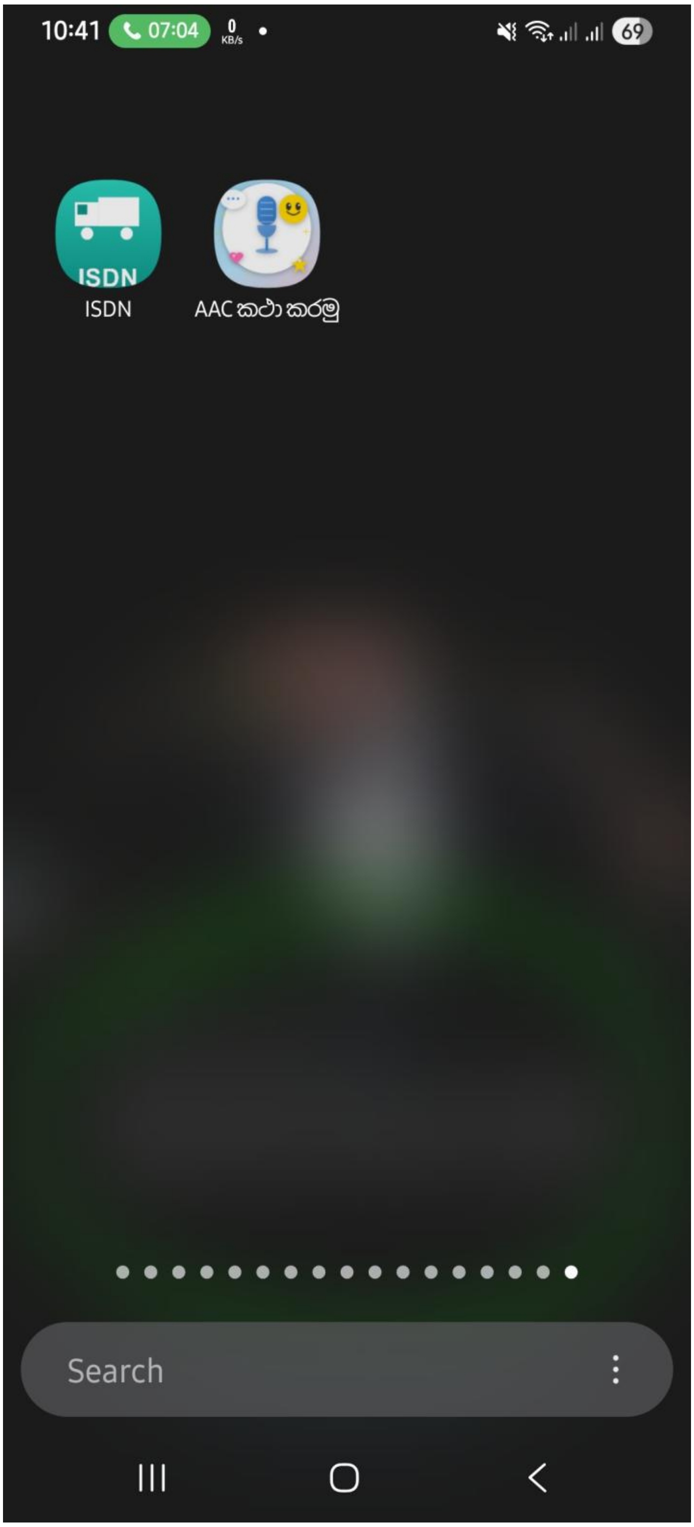
Figure 16: Application Running on Real Device

Tablet:



Phone:





The application performed acceptably on both devices, providing confidence that the prototype can function on real hardware before any wider pilot deployment.

3.12 Initial Field Exposure (Karapitiya Context)

An initial field exposure was conducted at Karapitiya Teaching Hospital in Galle to gain a practical understanding of the real-world context in which the proposed system would be used. This visit was exploratory in nature and did not involve formal data collection, as ethical approval had not yet been obtained at this stage.

Observations from this setting highlighted significant communication challenges faced by children with autism spectrum disorder. Many of the children present did not have reliable means of expressing basic needs such as hunger, discomfort, or emotional distress. Caregivers reported relying on continuous interpretation and guesswork, often without access to suitable digital tools. The absence of AAC applications supporting the Sinhala language was also identified as a critical gap, reinforcing the motivation for this project.

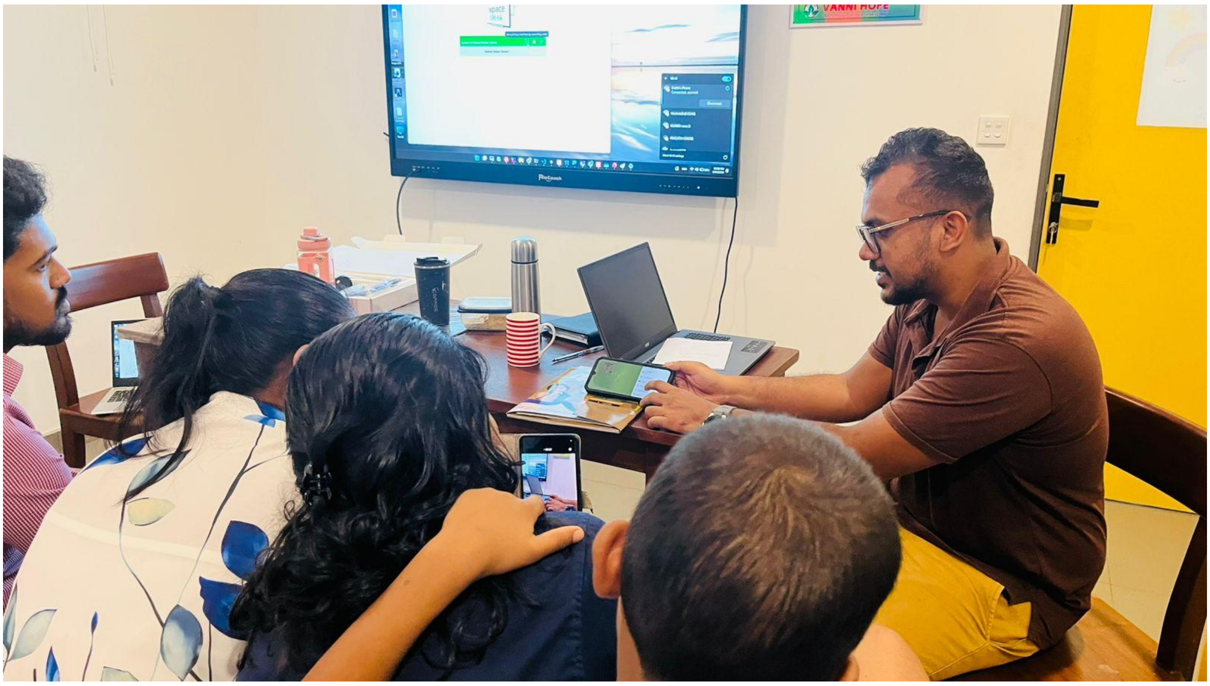
From an environmental perspective, the clinical setting presented several practical constraints. The ward environment was relatively noisy, attention spans among the children were limited, and many families did not have consistent access to tablets or similar devices. Internet connectivity in home environments was also reported as unreliable, even among families who owned compatible devices. These constraints were directly observed and had a clear influence on subsequent design decisions.

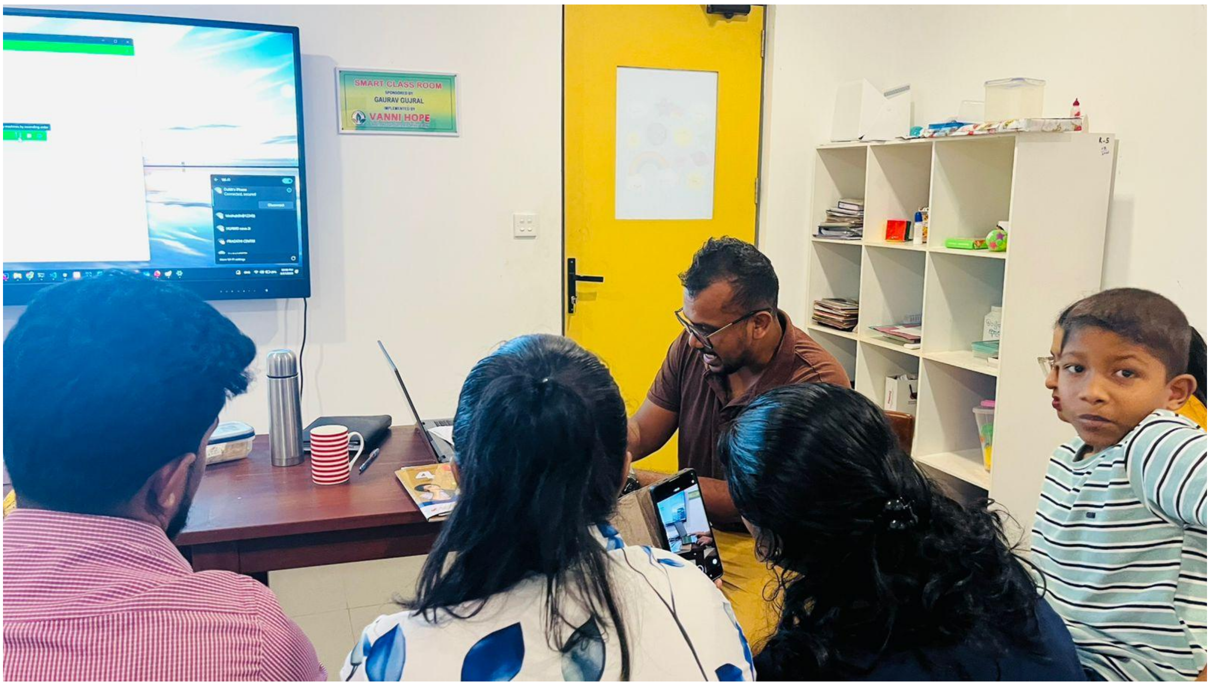
In particular, the adoption of an offline-first architecture, the implementation of a simplified user interface, and the decision to maintain a minimal and uncluttered symbol grid were all informed by insights gained during this field exposure.

Figure 17 illustrates the field exposure session conducted at Karapitiya Teaching Hospital, where the AAC application was demonstrated within a real-world context.

Drive Link Photos videos:- https://drive.google.com/drive/folders/1CoWy-is5cbDUR7BOddoymTZy5umPKJYY?usp=sharing

Figure 17: Field Exposure at Karapitiya Teaching Hospital





The formal pilot testing phase will be conducted following ethical approval. However, this initial exposure provided practical insights that could not be fully captured through the literature review alone and contributed significantly to shaping the overall system design.

The formal pilot testing phase will be conducted following ethical approval. However, this initial exposure provided practical insights that the literature review alone could not have fully captured, and it had a noticeable influence on the overall system design.

3.13 Supporting Links

The following links will be included in the final submission.

- **Google Colab (Model Training):** [To be provided in final submission]
- **Google Play Testing Link (Internal Testing):** [To be provided in final submission]
- **User Guidance (User Manual / Quick Guide):** [To be provided in final submission]

3.14 External Interest

There has been some informal interest in the project concept outside the academic context. Preliminary discussions have taken place with IdeaHub (Pvt) Ltd regarding the potential for releasing the application as a charitable initiative, particularly for the iOS platform where the developer account and distribution infrastructure would be provided through the company. However, no formal collaboration has been established at this stage. The project remains at the prototype stage, and Ministry of Health approval has not yet been obtained. Any external collaboration, charitable distribution, or outreach will only be pursued after system completion, pilot validation, and all necessary approvals are in place.

4. Further Work

The following subsections outline the remaining tasks required to complete the project. These tasks are aligned with the revised project plan presented in Section 5.6.

4.1 Platform A Completion

The remaining work for Platform A includes finalising the symbol set and resolving licensing issues, completing the full vocabulary in Sinhala, Tamil, and English with culturally appropriate symbols, integrating and evaluating text-to-speech for all three languages, implementing full customisation features (configurable grid size, user-added symbols, custom categories, colour themes, and font size settings), implementing visual scheduling functionality, and conducting internal usability testing with at least one therapist and one parent or caregiver.

Additional personalisation features are planned, including the ability for caregivers to record custom audio for symbols, edit existing cards, and add new symbols using photos from the device camera. These features are intended to improve engagement and communication effectiveness for children at ASD Levels 1–2.

4.2 Platform B and AI Integration

The remaining work for Platform B includes completing the curated dataset (2,000–5,000 images) combining public data with purpose-collected data, conducting full model training with hyperparameter tuning, achieving and documenting the 80% accuracy target with full confusion matrix analysis and per-class precision, recall, and F1 scores, integrating the TFLite model into the Flutter application with the face detection and preprocessing pipeline, implementing configurable emotion-adaptive logic and caregiver override, and testing emotion detection on multiple devices (Android and iOS).

4.3 Backend and Synchronisation

The remaining backend work includes completing manual export and import backup and restore flows, designing and implementing optional Firebase synchronisation with conflict handling (if pursued), and hardening security through role-based access, encryption, and audit logging.

4.4 Dashboard

The dashboard requires completion with progress visualisations, vocabulary management, and data export options. Feedback sessions with at least one therapist and one parent are also planned.

4.5 Pilot and Evaluation

The pilot phase requires obtaining ethics approval from Karapitiya Teaching Hospital and completing Ministry of Health processes, recruiting pilot participants (target of 5 to 15 children with ASD and their caregivers and therapists), conducting supervised pilot use over 4 to 8 weeks, collecting quantitative data (usage logs, emotion detection accuracy, task completion rates) and qualitative data (caregiver and therapist feedback), and analysing findings with documentation of limitations and recommendations.

4.6 Final Deliverables

Final deliverables include the final report incorporating pilot findings and full model evaluation, user documentation such as an installation guide and user manual, a deployment package containing the APK or IPA, model files, and backend configuration, and a presentation or demonstration for the examining panel.

Table 12: Remaining Work Plan

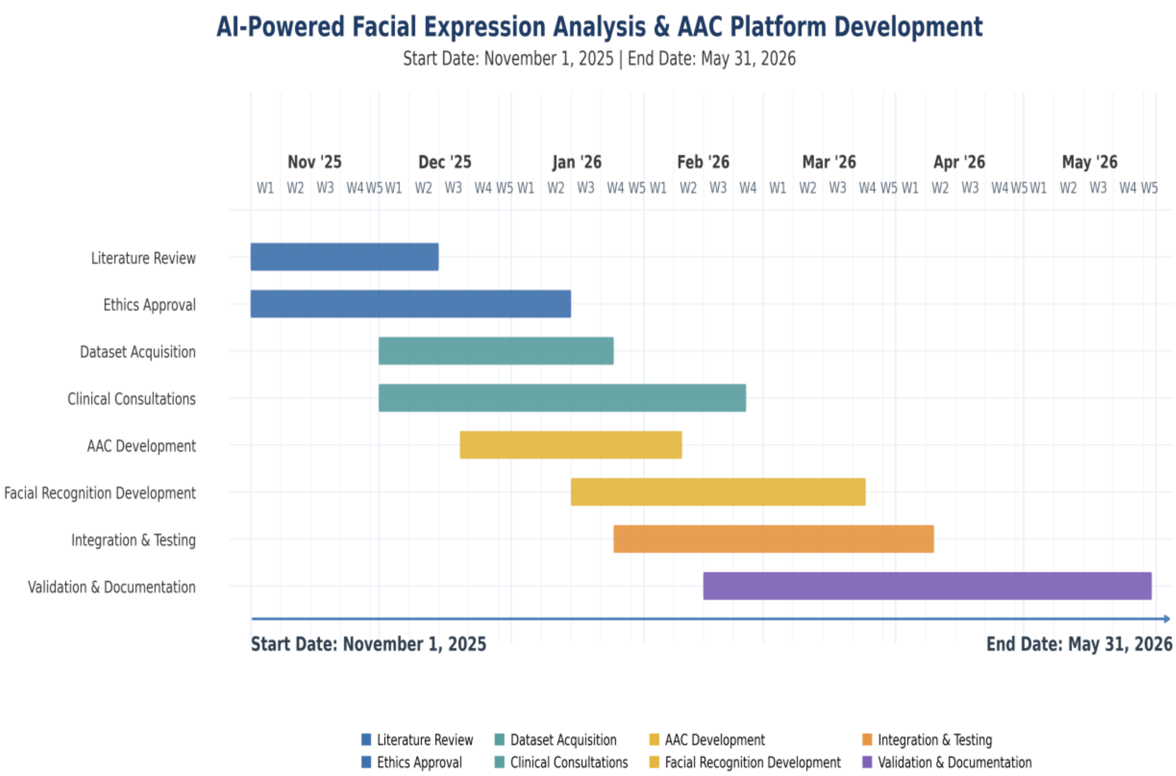
Task	Target Period	Dependency	Status
Finalise symbol set and licensing	March 2026	None	Pending
Complete Platform A (full trilingual AAC)	March 2026	Symbol set	Pending
Complete curated dataset	March to April 2026	Ethics approval	Pending
Full model training and evaluation	March to April 2026	Dataset	Pending
Platform B integration (FER + adaptation)	April 2026	Model	Pending
Full dashboard	April 2026	None	Pending
Full offline-first sync	April 2026	None	Pending
Ethics approval (hospital + MoH)	March to April 2026	Application	In preparation
Pilot recruitment	April 2026	Ethics approval	Pending
Pilot execution	May 2026	Recruitment	Pending
Final report	May 2026	All above	Pending

5. Progress Review

5.1 Original Project Plan

At the outset of the project, a Gantt chart was prepared as part of the project proposal to establish the planned schedule across the academic year. This chart divided the work into five sprints spanning approximately seven months, with each sprint building upon the deliverables of the previous one. Figure 18 presents the original project Gantt chart.

Figure 18: Initial Project Gantt Chart



The five sprints in the original plan were defined as follows.

Sprint 1 (November to December 2025) covered requirements analysis, architecture design, literature review, and the initial Flutter project setup. The expected deliverable was a documented system architecture and a basic application skeleton.

Sprint 2 (January to February 2026) covered Platform A core AAC features, including the symbol grid, trilingual support, basic TTS, and navigation. It also included Firebase backend configuration and initial dashboard implementation. The expected deliverable was a functional, though incomplete, AAC application and backend.

Sprint 3 (March 2026) covered dataset assembly, full model training and evaluation, TFLite export, and symbol set finalisation. The expected deliverable was a trained model ready for integration.

Sprint 4 (April 2026) covered Platform B FER integration, dashboard completion, ethics approval, pilot design, and participant recruitment. The expected deliverable was a complete system ready for pilot deployment.

Sprint 5 (May 2026) covered pilot execution, data collection and analysis, final report writing, and preparation of deliverables. The expected deliverable was the final report and all supporting documentation.

At the interim submission point, which falls at the end of February 2026, Sprints 1 and 2 were expected to be fully complete.

5.2 Progress Against the Original Plan

Sprint 1 (November to December 2025) was completed on schedule. The literature review, requirements analysis, system architecture, and technology stack selection were all delivered within the planned timeframe. The Flutter project was initialised with the intended folder structure and build targets for both Android and iOS. No significant issues arose during this phase.

Sprint 2 (January to February 2026) is substantially complete, though progress was uneven across different work packages. The core AAC features for Platform A, including the symbol grid, screen navigation, basic text-to-speech, and the multilingual switching framework, have been partially implemented and are functional at a prototype level. The AI model pipeline was set up ahead of schedule, with preliminary training experiments on public datasets completed during this sprint rather than in Sprint 3 as originally planned. This early start on the model work provides a useful buffer for the next phase.

However, four areas did not progress as quickly as the original plan anticipated.

The symbol set and licensing process proved more time-consuming than expected. Identifying symbols that are culturally appropriate for Sri Lankan users, and then navigating the licensing terms of various symbol libraries, required correspondence and research that had not been fully accounted for in the original estimate. This work remains in progress.

The Firebase backend is at an early prototype level. The decision was taken to prioritise the offline-first architecture and local storage layer over cloud integration. Firebase configuration has been started, but full synchronisation and authentication have been deliberately deferred. This was a conscious scope prioritisation rather than an unplanned delay.

Text-to-speech quality for Sinhala and Tamil fell short of expectations. The assumption that the default platform TTS engines would produce acceptable output proved incorrect during initial testing. Alternative TTS services will need to be evaluated in the next sprint.

The ethics application for Karapitiya Teaching Hospital has taken longer to coordinate than originally anticipated. The process involves both the hospital's institutional ethics committee and Ministry of Health governance requirements. The application is in advanced preparation but has not yet been formally submitted.

5.3 Justification for Delays

The delays identified in Section 5.2 are minor and have specific, identifiable causes. Each is explained below with its impact and the corrective action taken.

Delay 1: Symbol Set Licensing

- **Problem.** Most established AAC symbol libraries (such as PCS and Widgit) are designed for Western contexts. The food, clothing, and cultural symbols in these libraries do not represent Sri Lankan daily life.
- **Cause.** Finding and licensing culturally appropriate symbols required correspondence with multiple symbol library providers and manual adaptation of existing assets. The time required for this process was significantly underestimated in the original plan.
- **Impact.** The full trilingual vocabulary for Platform A is not yet complete.
- **Action taken.** Manual sourcing of culturally relevant symbols is ongoing. This task has been carried forward to Sprint 3 as a priority item.

Delay 2: Ethics Coordination

- **Problem.** The formal ethics application for the pilot study at Karapitiya Teaching Hospital has not yet been submitted.
- **Cause.** The approval process involves coordination with both the hospital's institutional ethics committee and the Ministry of Health. The administrative timelines and procedural requirements for these bodies were not fully known at the planning stage and have proved longer than initially assumed.
- **Impact.** Purpose-collected data from children with ASD cannot yet be used for model training, and the formal pilot study cannot begin until approval is obtained.
- **Action taken.** The ethics application is in advanced preparation. An exploratory field exposure was conducted in the meantime to inform design decisions without formal data collection.

Delay 3: Text-to-Speech Quality

- **Problem.** The default platform TTS engines on Android do not produce acceptable speech output for Sinhala and Tamil.
- **Cause.** The original plan assumed that the built-in TTS engines would provide adequate quality for all three languages. This assumption was only disproved during implementation when the Sinhala and Tamil output was tested and found to be unnatural and difficult to understand.
- **Impact.** Trilingual TTS is not yet functional. English TTS works as expected.
- **Action taken.** Evaluation of third-party TTS services and recorded audio alternatives has been added to the Sprint 3 backlog.

Delay 4: AI Model Training on Google Colab

- **Problem.** The pace of model experimentation was slower than originally anticipated.
- **Cause.** The training pipeline was executed on Google Colab using the free-tier GPU allocation. The free tier imposes session time limits, which meant that longer training runs were sometimes interrupted before completion, resulting in lost checkpoints and the need to repeat experiments.
- **Impact.** Fewer training iterations were completed than planned during Sprint 2, though preliminary results were still obtained.
- **Action taken.** More frequent checkpoint saving was adopted to prevent further data loss. The backlog for Sprint 3 includes completing full model training with the final dataset.

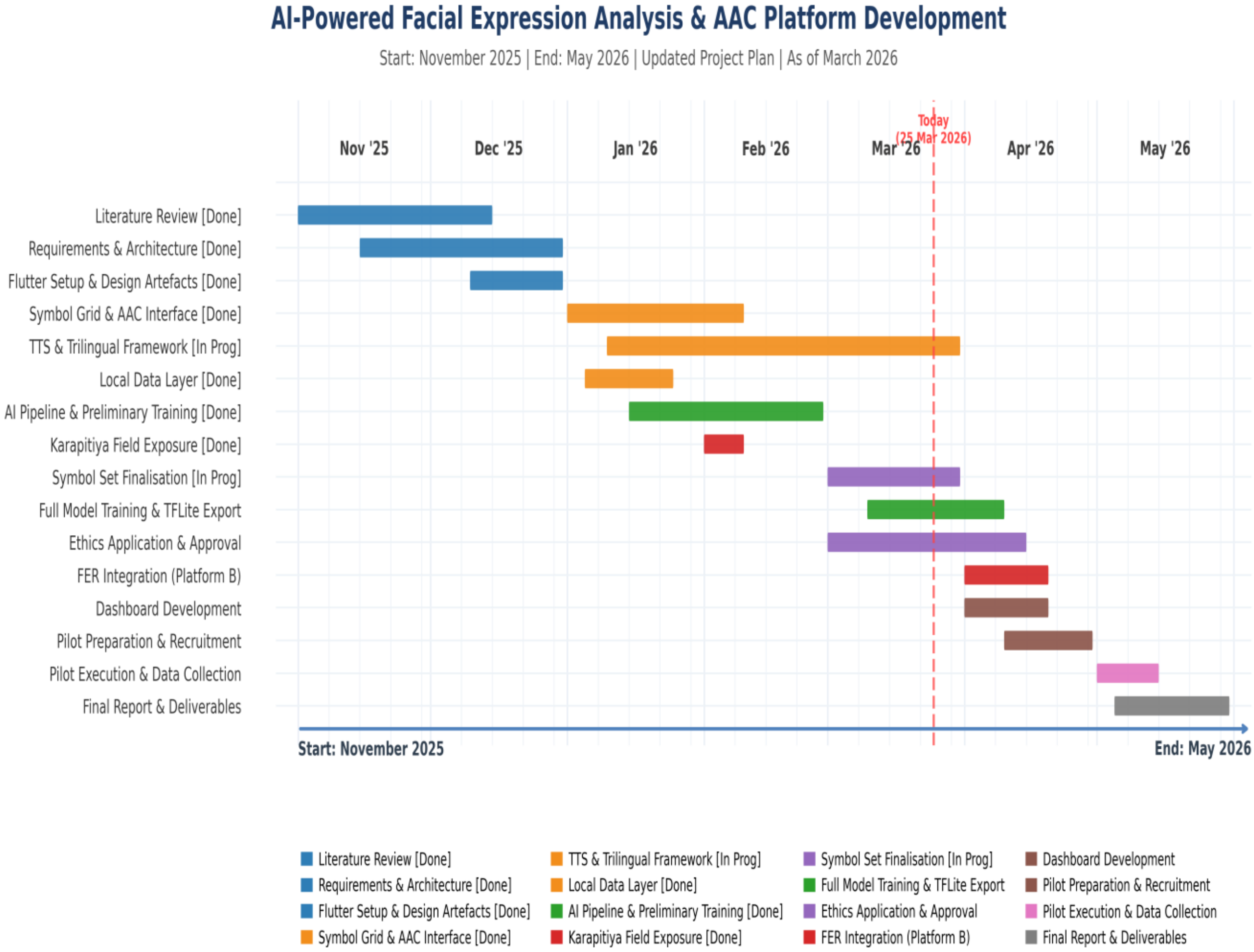
Overall Impact Assessment

None of these delays affect the critical path in a way that compromises the final deadline, provided the revised plan outlined in Section 5.4 is followed. The critical path runs through ethics approval, dataset collection, model training, Platform B integration, pilot execution, and the final report. Platform A development and model training on public data can proceed independently of the ethics timeline.

5.4 Revised Plan for Remaining Work

Based on the progress to date and the delays identified, the project plan has been revised for the remaining sprints. The key adjustment is the introduction of an overlap between Sprints 3 and 4, where Platform A completion, model training, and ethics preparation proceed in parallel. This is feasible because these tasks have limited interdependencies and Scrum allows the backlog to be reprioritised between sprints as needed. Figure 19 presents the updated project Gantt chart reflecting these changes.

Figure 19: Updated Project Gantt Chart



The revised schedule is as follows.

March 2026 (Revised Sprint 3). The focus during this period will be on finalising the symbol set, completing Platform A vocabulary and TTS integration, submitting the ethics application, continuing model training on public datasets, and beginning dataset curation.

April 2026 (Revised Sprint 4). This period will focus on full model training completion, Platform B FER integration and testing, dashboard completion, and ethics follow-up. Pilot protocol design and participant recruitment will also begin during this sprint.

May 2026 (Revised Sprint 5 and Final Phase). This final sprint covers pilot execution at Karapitiya Teaching Hospital, data collection and analysis, final report writing, preparation of deliverables, and the project presentation.

The main difference between the original and revised plans is the parallel scheduling of previously sequential tasks during March to May 2026 and the compression of Sprints 3 to 5 into one-month cycles. This approach absorbs the minor delays without extending the overall project timeline.

Throughout the project, sprint progress and task status were tracked using a Google Sheets spreadsheet, which served as both the product backlog and the sprint backlog tracker. Figure 20 presents the sprint tracking spreadsheet used during the project.

Figure 20: Google Sheets Sprint Tracking Spreadsheet

Link:https://docs.google.com/spreadsheets/d/1m2-U_Ga_bLSauQyvLuXoFvqTGAIyuhJ-/edit?usp=sharing&ouid=106973302403748897349&rtpof=true&sd=true

AAC System - Product Backlog

Project: AI-Powered AAC System for Children with ASD (Sinhala/Tamil/English)

#	Backlog Item	Priority	Target Sprint
1	Literature review and requirements analysis	High	Sprint 1
2	System architecture and technology selection	High	Sprint 1
3	Flutter project setup (Android and iOS targets)	High	Sprint 1
4	Symbol grid, navigation, and basic AAC flow	High	Sprint 2
5	Trilingual UI framework (Sinhala, Tamil, English)	High	Sprint 2
6	Basic TTS integration	Medium	Sprint 2
7	Firebase project setup (exploratory)	Medium	Sprint 2
8	AI model pipeline setup and initial training	High	Sprint 2-3
9	Full model training and TFLite export	High	Sprint 3
10	Platform B FER integration	High	Sprint 3
11	Emotion-adaptive vocabulary logic	Medium	Sprint 3
12	Dashboard completion	Medium	Sprint 4
13	Ethics application and approval	High	Sprint 4
14	Pilot protocol and participant recruitment	Medium	Sprint 4
15	Pilot execution and data collection	High	Sprint 5
16	Final report and deliverables	High	Sprint 5

A1 | AAC System - Sprint Progress Summary

AAC System - Sprint Progress Summary

Sprint	Period	Total Tasks	Done	In Progress	Pending	Completion %
Sprint 1	Nov - Dec 2025	6	6	0	0	100%
Sprint 2	Jan - Feb 2026	10	10	0	0	100%
Sprint 3	Mar 2026	7	5	2	0	71%
Sprint 4	Apr 2026	5	3	2	0	60%
Sprint 5	May 2026	5	1	1	3	20%

Overall Progress: 25 of 33 tasks completed (76%). Platform B integration and pilot-readiness activi

the workbook used for sprint tracking was organised into several sheets so that backlog management, day-to-day progress, and governance could be recorded in one place. The Product Backlog sheet listed prioritised features and epics. The Sprint Backlogs sheet broke work down by sprint. The Sprint Tracker sheet followed a Kanban-style layout for task status. Additional sheets supported project oversight, namely Sprint Summary for high-level progress, Milestones for key dates, Meeting Log for supervisor and stakeholder reviews, Issue Tracker for defects and blockers, Decision Log for agreed changes, Retrospectives for sprint reflections, and Time Log for effort records. The same structure was maintained when the file was prepared for upload to Google Drive and reviewed as a screenshot for Figure 20.

The Sprint Summary sheet reports aggregate progress across the full product backlog (for example, 25 of 33 tasks completed, approximately 76 per cent). This figure reflects backlog completion across all planned sprints and includes completed early-start items brought forward from later sprint plans. Tasks are marked as complete only where implementation evidence is available, while partially completed activities are retained as in progress to avoid overstating delivery. The remaining items are concentrated in pilot execution, final analysis, and end-stage documentation tasks scheduled toward the final submission window.

5.5 Risk Assessment for Remaining Work

The most significant risk to the revised plan is a delay in ethics approval, which would prevent purpose collected data from being included in the training dataset and would delay the pilot study. The primary contingency is to complete model training and evaluation using public datasets only, and to conduct Platform A usability testing with informal participants rather than a formal pilot. This would still produce a valid proof of concept, though with acknowledged limitations.

Other risks, including model accuracy below the 80 per cent target, insufficient TTS quality for Sinhala and Tamil, and limited therapist or parent availability for the pilot, are mitigated through the strategies documented in Section 2.10.

5.6 Work Breakdown Structure

The project is divided into the following main work packages.

Requirements and Design covers the literature review, stakeholder analysis, requirements gathering, system architecture design, and technology selection.

Platform A Development covers Flutter setup, symbol grid implementation, trilingual support, text to speech integration, customisation features, visual scheduling, and usage logging.

AI Model covers data sourcing, preprocessing, transfer learning setup, model training, evaluation, quantisation, and benchmarking.

Platform B Integration covers face detection, TFLite inference, emotion adaptive logic, caregiver override, and end to end testing.

Backend and Dashboard covers Firebase configuration, schema design, security considerations, offline synchronisation handling, dashboard development, and progress visualisation.

Ethics and Pilot covers consent forms, translations, the ethics application, Ministry coordination, participant recruitment, pilot execution, and analysis.

Documentation covers the interim report, final report, user documentation, presentation materials, and code archival.

5.7 Conclusion

The project has completed Sprint 1 in full and the majority of Sprint 2. This section summarises the progress made against each of the six project objectives and outlines the evaluation work that remains before the objectives can be fully achieved.

Objective 1 was to design a dual-platform system architecture with trilingual support, offline-first operation, and secure data management. This objective has been substantially achieved. The system architecture, including the dual-platform design (Platform A for ASD Level 1-2 and Platform B for Level 3+), the offline-first data layer, and the technology stack, has been fully designed and documented. The architecture diagram, ER diagram, use case diagram, and class diagram have all been produced as design artefacts. What remains is the implementation of optional Firebase synchronisation and the completion of security hardening, both of which are planned for later sprints.

Objective 2 was to implement a cross-platform mobile application using Flutter. This objective is partially achieved. The Flutter project has been set up with build targets for both Android and iOS. Core AAC features, including the symbol grid, category navigation, multilingual switching framework, and basic text-to-speech, are functional at a prototype level. The application has been deployed to the Google Play Console under the Internal Testing track and tested on physical Android devices. Platform A completion (full trilingual vocabulary, customisation features, visual scheduling) and iOS testing remain outstanding.

Objective 3 was to train and deploy an EfficientNetB0-based FER model targeting at least 80% accuracy across six emotion classes. This objective is in progress. The AI model pipeline has been set up on Google Colab, preliminary training experiments have been completed using public datasets, and the model architecture has been finalised (EfficientNetB0 with CBAM, after MobileNetV2 was evaluated and replaced). Current results show training accuracy of 85-87% and validation accuracy of 82-84%, though test accuracy remains lower at approximately 66%. Full model training on the complete dataset, TFLite export, and on-device integration are planned for Sprint 3. Closing the generalisation gap between validation and test accuracy is the primary model objective going forward.

Objective 4 was to develop a therapist-parent dashboard. This objective is at an early stage. A basic dashboard prototype has been started with login UI, navigation, and placeholder progress views. Full dashboard development, including progress visualisations, vocabulary management, and data export, is planned for Sprint 4.

Objective 5 was to establish ethical approval protocols and a clinical partnership for a pilot study at Karapitiya Teaching Hospital. This objective is in progress. Initial contact with Karapitiya Teaching Hospital has been established, and an exploratory field exposure was conducted to understand the clinical context. Draft consent forms and participant information sheets have been prepared in English (included in Appendices A and B). The formal ethics application is in advanced preparation but has not yet been submitted. Ministry of Health approval requirements have been researched and documented.

Objective 6 was to document the project in accordance with FC6P01ES module requirements. This objective is on track. The interim report has been prepared using Harvard referencing, with all sections structured according to the module template.

Remaining Work and Evaluation

The core development work for Sprints 1 and 2 has established the foundation of the system, but the most critical phase of the project lies ahead. The primary remaining task is the formal evaluation of the system, which is essential for determining whether the project objectives have been fully met. The evaluation will follow the mixed-methods approach described in Section 2.7. Quantitatively, the FER model will be evaluated against the 80% accuracy target using precision, recall, F1-score, and a confusion matrix, and system usability will be measured through structured questionnaires and the System Usability Scale (SUS) administered to caregivers and therapists. Qualitatively, semi-structured interviews and observational notes from the pilot study at Karapitiya Teaching Hospital will capture the practical experiences of users and identify areas for improvement.

It is through this evaluation that the project objectives will ultimately be validated or refined. The quantitative metrics will determine whether the system meets its defined performance and usability thresholds, while the qualitative findings will reveal whether the system is genuinely useful and appropriate in the Sri Lankan clinical and home context. The conclusions in the final report will be drawn by triangulating these two streams of evidence, as described in Section 2.7.5.

Risks and Contingency

The most significant risk is a delay in ethics approval, which would prevent purpose-collected data from being included in the training set and would push back the pilot study. A contingency plan exists for this scenario. Model training and evaluation would proceed using public datasets only, and Platform A usability testing would be conducted with informal participants. This would still produce a valid proof of concept, though with acknowledged limitations in the evaluation.

Some tasks have taken longer than the original plan allowed for, particularly symbol set licensing, ethics coordination, and the discovery that platform TTS engines do not produce acceptable Sinhala and Tamil output. These delays are acknowledged but none are severe enough to compromise the final deadline, provided the revised plan (Section 5.4) is followed.

Summary

At this stage, the project is in a reasonable position relative to the academic timeline. The foundational architecture, the development environment, the preliminary AI model experiments, and the clinical partnership groundwork are all in place. The remaining sprints focus on completing the implementation, conducting the formal evaluation, and drawing evidence-based conclusions on whether the system effectively addresses the identified gap in AAC provision for children with autism in Sri Lanka.

6. References

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Mane, R., Monga, R., Moore, S., Murray, D.G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y. and Zheng, X. (2016) 'TensorFlow: a system for large-scale machine learning', in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*. Berkeley, CA: USENIX Association, pp. 265-283.

Alant, E. and Bornman, J. (2021) *Augmentative and alternative communication: engagement and participation*. San Diego, CA: Plural Publishing.

American Psychiatric Association (2013) *Diagnostic and statistical manual of mental disorders*. 5th edn. Washington, DC: American Psychiatric Publishing.

American Speech-Language-Hearing Association (2022) *Augmentative and alternative communication (AAC)*. Available at: https://www.asha.org/public/speech/disorders/aac/ (Accessed: 22 February 2025).

Barrett, L.F., Adolphs, R., Marsella, S., Martinez, A.M. and Pollak, S.D. (2019) 'Emotional expressions reconsidered: challenges to inferring emotion from human facial movements', *Psychological Science in the Public Interest*, 20(1), pp. 1-68.

Beukelman, D.R. and Light, J.C. (2020) *Augmentative and alternative communication: supporting children and adults with complex communication needs*. 5th edn. Baltimore, MD: Paul H. Brookes.

Boehm, B.W. (1988) 'A spiral model of software development and enhancement', *Computer*, 21(5), pp. 61-72.

Brooke, J. (1996) 'SUS: a "quick and dirty" usability scale', in Jordan, P.W., Thomas, B., Weerdmeester, B.A. and McClelland, I.L. (eds.) *Usability Evaluation in Industry*. London: Taylor and Francis, pp. 189-194.

Creswell, J.W. and Creswell, J.D. (2018) *Research design: qualitative, quantitative, and mixed methods approaches*. 5th edn. Thousand Oaks, CA: SAGE Publications.

Dawe, M. (2006) 'Desperately seeking simplicity: how young adults with cognitive disabilities and their families adopt assistive technologies', in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. New York: ACM, pp. 1143-1152.

Department of Census and Statistics, Sri Lanka (2012) *Census of population and housing - 2012*. Colombo: Department of Census and Statistics.

Divan, G., Vajaratkar, V., Desai, M.U., Strik-Lievers, L. and Patel, V. (2021) 'Prevalence and risk factors for autism spectrum disorder in low- and middle-income countries: a systematic review and meta-analysis', *Global Mental Health*, 8, e30.

Ekman, P. and Friesen, W.V. (1971) 'Constants across cultures in the face and emotion', *Journal of Personality and Social Psychology*, 17(2), pp. 124-129.

Elsabbagh, M., Divan, G., Koh, Y.J., Kim, Y.S., Kauchali, S., Marcin, C., Montiel-Nava, C., Patel, V., Paula, C.S., Wang, C., Yasamy, M.T. and Fombonne, E. (2012) 'Global prevalence of autism and other pervasive developmental disorders', *Autism Research*, 5(3), pp. 160-179.

Firebase (2023) *Firebase documentation*. Available at: https://firebase.google.com/docs (Accessed: 15 January 2025).

Fletcher-Watson, S. and Happe, F. (2019) *Autism: a new introduction to psychological theory and current debate*. 2nd edn. Abingdon: Routledge.

Floridi, L., Cowls, J., Beltrametti, M., Chatila, R., Chazerand, P., Dignum, V., Luetge, C., Madelin, R., Pagallo, U., Rossi, F., Schafer, B., Valcke, P. and Vayena, E. (2018) 'AI4People: an ethical framework for a good AI society', *Minds and Machines*, 28(4), pp. 689-707.

Flutter (2023) *Flutter documentation*. Available at: https://flutter.dev/docs (Accessed: 15 January 2025).

Ganz, J.B. (2015) *AAC for individuals with autism spectrum disorders*. New York: Springer.

Ganz, J.B., Davis, J.L., Lund, E.M., Goodwyn, F.D. and Simpson, R.L. (2012) 'A meta-analysis of single case research studies on aided augmentative and alternative communication systems with individuals with autism spectrum disorders', *Journal of Autism and Developmental Disorders*, 42(1), pp. 60-74.

Goodfellow, I., Bengio, Y. and Courville, A. (2015) *Deep learning*. Cambridge, MA: MIT Press.

Grossard, C., Dapogny, A., Cohen, D., Bernheim, S., Martinerie, J., Janvier, M., Grynszpan, O., Chaby, L., Bailly, K. and Dubuisson, S. (2020) 'Children with autism spectrum disorder produce more ambiguous and less socially meaningful facial expressions', *Molecular Autism*, 11(1), p. 5.

Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H. (2017) 'MobileNets: efficient convolutional neural networks for mobile vision applications', *arXiv preprint arXiv:1704.04861*.

Howard, A., Sandler, M., Chen, B., Wang, W., Chen, L.C., Tan, M., Chu, G., Vasudevan, V., Zhu, Y., Pang, R., Adam, H. and Le, Q. (2019) 'Searching for MobileNetV3', in *Proceedings of the IEEE/CVF International Conference on Computer Vision*. Piscataway, NJ: IEEE, pp. 1314-1324.

International Telecommunication Union (2022) *Measuring digital development: facts and figures 2022*. Geneva: ITU.

Ioffe, S. and Szegedy, C. (2015) 'Batch normalization: accelerating deep network training by reducing internal covariate shift', in *Proceedings of the 32nd International Conference on Machine Learning*. Lille, France: JMLR, pp. 448-456.

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H. and Kalenichenko, D. (2018) 'Quantization and training of neural networks for efficient integer-arithmetic-only inference', in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. Piscataway, NJ: IEEE, pp. 2704-2713.

Jobin, A., Ienca, M. and Vayena, E. (2019) 'The global landscape of AI ethics guidelines', *Nature Machine Intelligence*, 1(9), pp. 389-399.

Kothari, C.R. (2004) *Research methodology: methods and techniques*. 2nd edn. New Delhi: New Age International Publishers.

LeCun, Y., Bengio, Y. and Hinton, G. (2015) 'Deep learning', *Nature*, 521(7553), pp. 436-444.

Li, S. and Deng, W. (2020) 'Deep facial expression recognition: a survey', *IEEE Transactions on Affective Computing*, 13(3), pp. 1195-1215.

Light, J.C. and McNaughton, D. (2012) 'The changing face of augmentative and alternative communication: past, present, and future challenges', *Augmentative and Alternative Communication*, 28(4), pp. 197-204.

Light, J.C. and McNaughton, D. (2015) 'Designing AAC research and intervention to improve outcomes for individuals with complex communication needs', *Augmentative and Alternative Communication*, 31(2), pp. 85-96.

Lord, C., Brugha, T.S., Charman, T., Cusack, J., Dumas, G., Frazier, T., Jones, E.J.H., Jones, R.M., Pickles, A., State, M.W., Taylor, J.L. and Veenstra-VanderWeele, J. (2020) 'Autism spectrum disorder', *Nature Reviews Disease Primers*, 6(1), p. 5.

Lorah, E.R., Parnell, A., Whitby, P.S. and Hantula, D. (2015) 'A systematic review of tablet computers and portable media players as speech generating devices for individuals with autism spectrum disorder', *Journal of Autism and Developmental Disorders*, 45(12), pp. 3792-3804.

Maenner, M.J., Warren, Z., Williams, A.R. et al. (2023) 'Prevalence and characteristics of autism spectrum disorder among children aged 8 years', *MMWR Surveillance Summaries*, 72(2), pp. 1-14.

Mazefsky, C.A., Herrington, J., Siegel, M., Scarpa, A., Maddox, B.B., Scahill, L. and White, S.W. (2013) 'The role of emotion regulation in autism spectrum disorder', *Journal of the American Academy of Child and Adolescent Psychiatry*, 52(7), pp. 679-688.

McNaughton, D. and Light, J. (2013) 'The iPad and mobile technology revolution: benefits and challenges for individuals who require augmentative and alternative communication', *Augmentative and Alternative Communication*, 29(2), pp. 107-116.

Millar, D.C., Light, J.C. and Schlosser, R.W. (2006) 'The impact of augmentative and alternative communication intervention on the speech production of individuals with developmental disabilities', *Journal of Speech, Language, and Hearing Research*, 49(2), pp. 248-264.

Ministry of Health, Sri Lanka (2020) *National guideline for ethics review of health research in Sri Lanka*. Colombo: Ministry of Health.

Nair, V. and Hinton, G.E. (2010) 'Rectified linear units improve restricted Boltzmann machines', in *Proceedings of the 27th International Conference on Machine Learning*. Madison, WI: Omnipress, pp. 807-814.

Perera, H., Wijewardena, K., Aluthwelage, R., Seneviratne, S. and Buddhika, K. (2019) 'Prevalence of autism spectrum disorder in Sri Lanka: a population-based study', *Sri Lanka Journal of Child Health*, 48(2), pp. 133-138.

Picard, R.W. (2000) *Affective computing*. Cambridge, MA: MIT Press.

Romski, M. and Sevcik, R.A. (2005) 'Augmentative communication and early intervention: myths and realities', *Infants and Young Children*, 18(3), pp. 174-185.

Samad, A., Razick, S., De Silva, M.V.C. and Hettiarachchi, S. (2020) 'Challenges and opportunities for autism services in Sri Lanka', *Journal of Autism and Developmental Disorders*, 50(8), pp. 3023-3029.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.C. (2018) 'MobileNetV2: inverted residuals and linear bottlenecks', in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. Piscataway, NJ: IEEE, pp. 4510-4520.

Shorten, C. and Khoshgoftaar, T.M. (2019) 'A survey on image data augmentation for deep learning', *Journal of Big Data*, 6(1), p. 60.

Sommerville, I. (2016) *Software engineering*. 10th edn. Harlow: Pearson Education.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) 'Dropout: a simple way to prevent neural networks from overfitting', *Journal of Machine Learning Research*, 15(1), pp. 1929-1958.

TensorFlow (2023) *TensorFlow Lite documentation*. Available at: https://www.tensorflow.org/lite (Accessed: 15 January 2025).

Trevisan, D.A., Hoskyn, M. and Birmingham, E. (2018) 'Facial expression production in autism: a meta-analysis', *Autism Research*, 11(12), pp. 1586-1601.

Wickramasinghe, N., Dissanayake, A. and Samarasinghe, D. (2021) 'Parent training and support programmes for autism in low-resource settings: a scoping review', *Global Health Action*, 14(1), 1910556.

World Health Organization (2021) *Autism spectrum disorders*. Available at: https://www.who.int/news-room/fact-sheets/detail/autism-spectrum-disorders (Accessed: 22 February 2025).

World Medical Association (2013) 'World Medical Association Declaration of Helsinki: ethical principles for medical research involving human subjects', *JAMA*, 310(20), pp. 2191-2194.

Yosinski, J., Clune, J., Bengio, Y. and Lipson, H. (2014) 'How transferable are features in deep neural networks?', in *Advances in Neural Information Processing Systems*, 27. Red Hook, NY: Curran Associates, pp. 3320-3328.

7. Bibliography

The following sources were consulted during preparation of this report and have informed the background, methodology, or discussion.

Bishop, C.M. (2006) *Pattern recognition and machine learning*. New York: Springer.

Chollet, F. (2017) *Deep learning with Python*. Shelter Island, NY: Manning Publications.

Deng, J., Dong, W., Socher, R., Li, L.J., Li, K. and Fei-Fei, L. (2009) 'ImageNet: a large-scale hierarchical image database', in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. Piscataway, NJ: IEEE, pp. 248-255.

Ekman, P. and Friesen, W.V. (1978) *Facial Action Coding System: a technique for the measurement of facial movement*. Palo Alto, CA: Consulting Psychologists Press.

He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep residual learning for image recognition', in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. Piscataway, NJ: IEEE, pp. 770-778.

Kaggle (2013) *FER-2013: Facial Expression Recognition 2013 Dataset*. Available at: https://www.kaggle.com/datasets/msambare/fer2013 (Accessed: 15 January 2025).

Mollahosseini, A., Hasani, B. and Mahoor, M.H. (2019) 'AffectNet: a database for facial expression, valence, and arousal computing in the wild', *IEEE Transactions on Affective Computing*, 10(1), pp. 18-31.

Pressman, R.S. and Maxim, B.R. (2019) *Software engineering: a practitioner's approach*. 9th edn. New York: McGraw-Hill Education.

Simonyan, K. and Zisserman, A. (2015) 'Very deep convolutional networks for large-scale image recognition', in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. San Diego, CA: ICLR.

End of Interim Report

Appendix A: Platform A (Level 1-2) User Interface and Features

This appendix presents additional user interface screens and implementation details related to Platform A, which is designed for children at Autism Spectrum Disorder (ASD) Level 1-2.

Platform A provides a customisable, symbol-based/Image Augmentative and Alternative Communication (AAC) interface with trilingual support (Sinhala, Tamil, and English). It is intended for children who possess some level of functional communication but benefit from structured visual support to enhance interaction.

The separation of the system into Platform A and Platform B was informed by clinical observations and initial field exposure at Karapitiya Teaching Hospital. Feedback from clinicians indicated that children at lower ASD severity levels require a simpler, highly customisable interface, while children at higher severity levels (Level 3 and above) require additional assistive mechanisms such as emotion recognition.

Although Platform A is an important component of the overall system, the primary research focus of this study is on Platform B, which integrates facial expression recognition (FER) to support children with severe communication impairments. Therefore, detailed figures and experimental results related to Platform B are presented in the main body of the report, while supporting visual evidence for Platform A is included in this appendix to maintain clarity and focus.

The following figures illustrate the key user interface components and interaction flow of Platform A.

Figure A1: Platform A - Home Scree

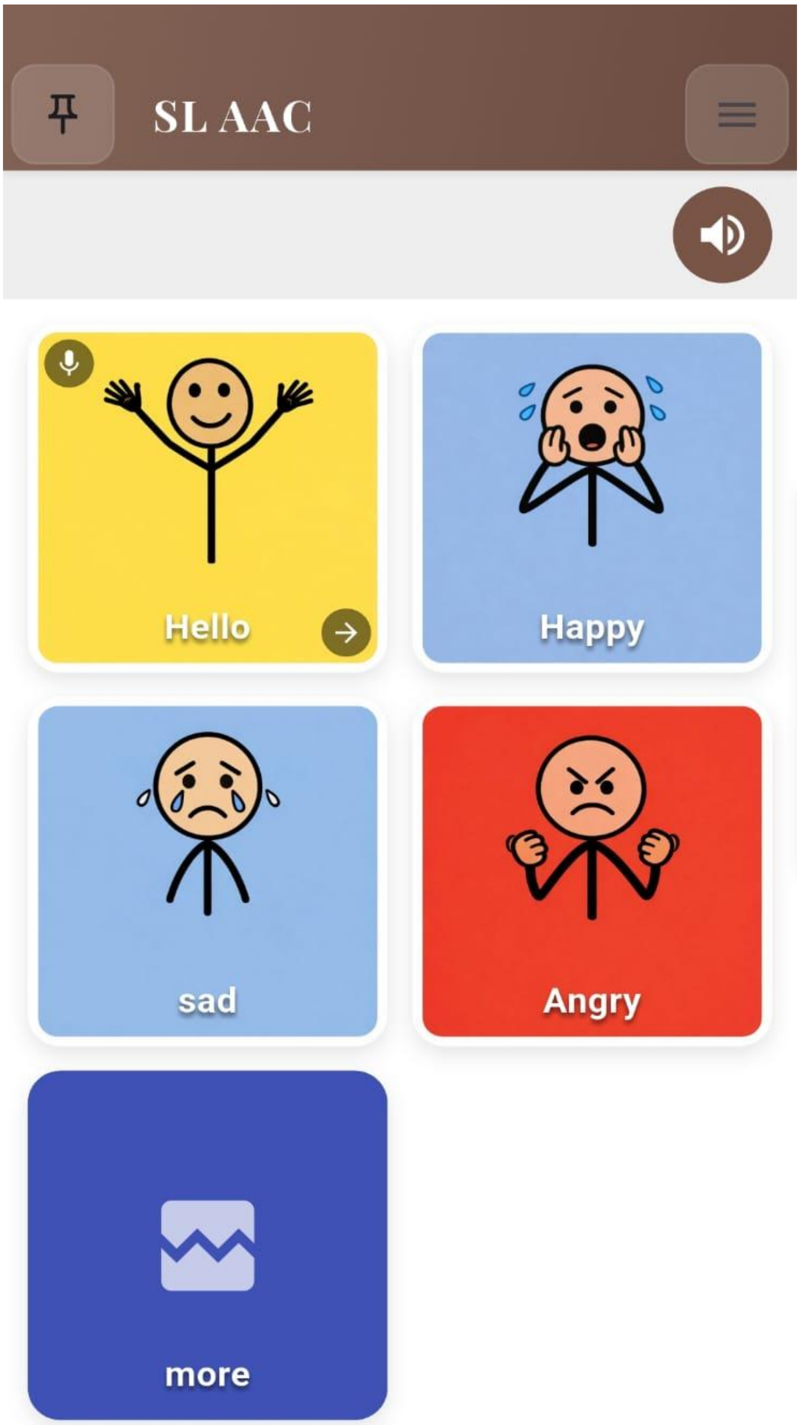


Figure A2: Platform A - Category Selection Interface

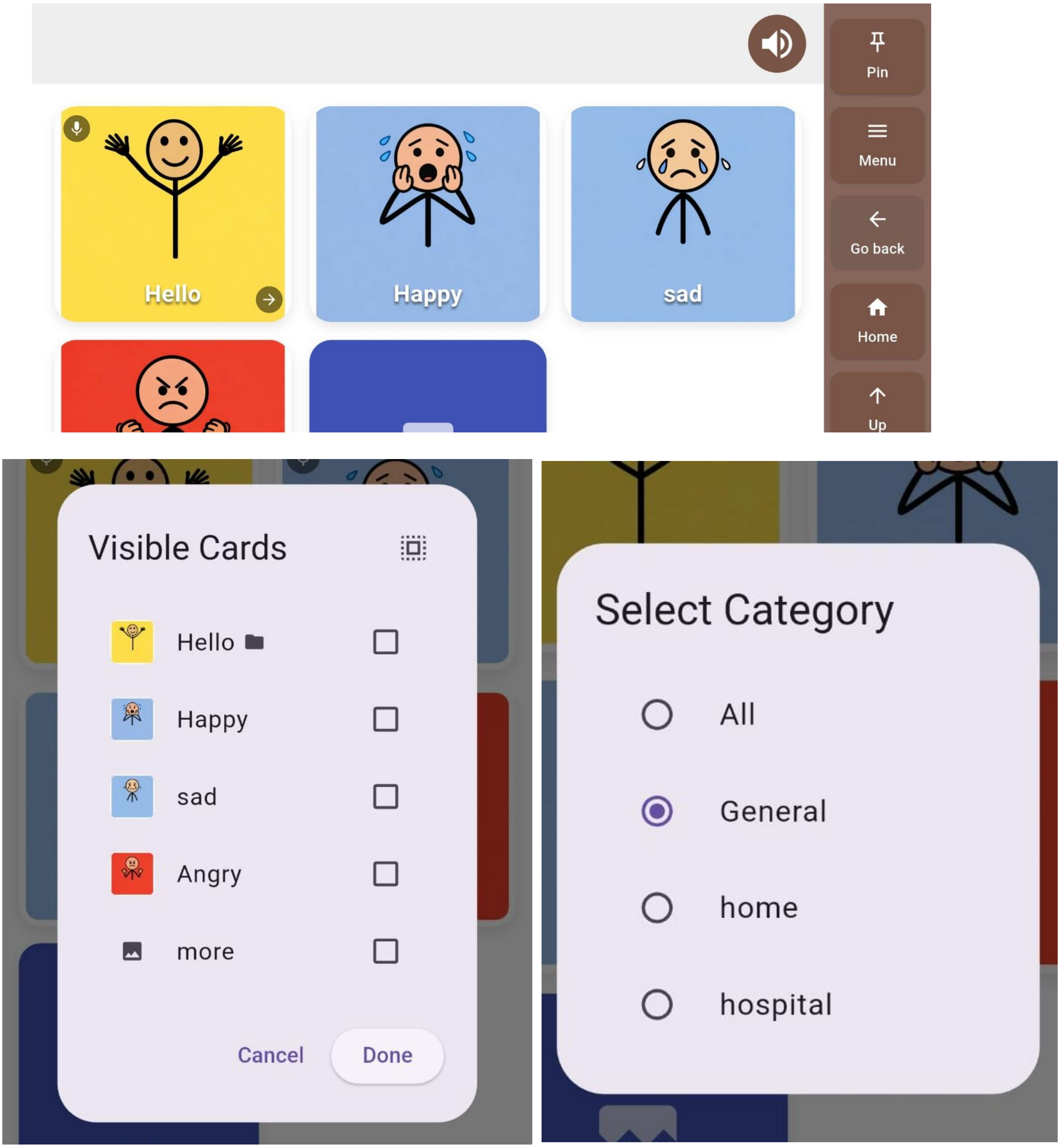


Figure A3: Platform A - Capture photo and record parent voices

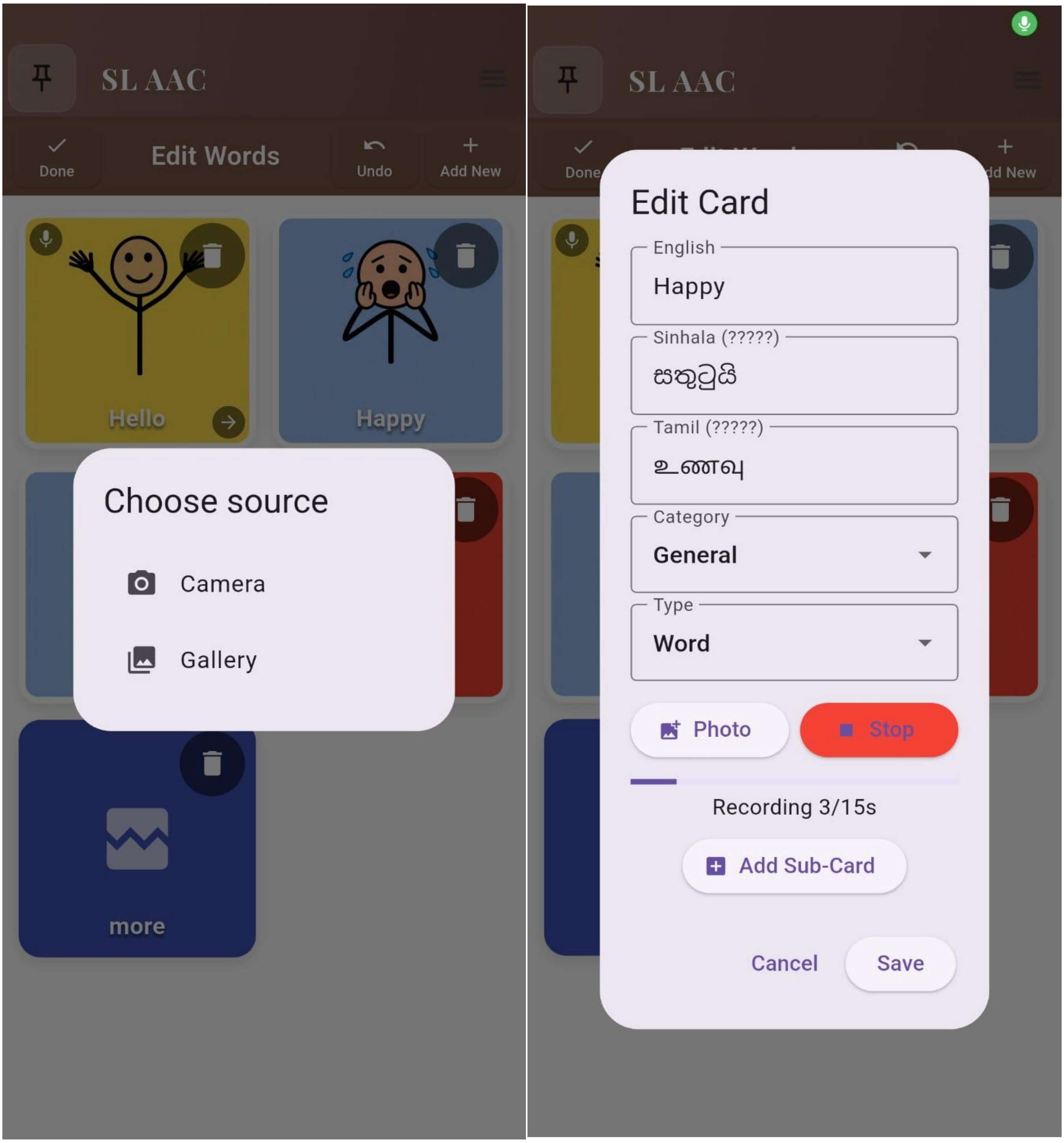


Figure A4: Platform A - Settings and Customisation Screen